scientific reports

Check for updates

OPEN

Brute-force attack mitigation on remote access services via software-defined perimeter

Francis A. Ruambo[1,2,3], Elijah E. Masanga[3], Beston Lufyagila[2], Abdelhamied A. Ateya[4,5], Ahmed A. Abd El-Latif[4,6,7]✉, May Almousa[8] & Bassem Abd-El-Atty[9]✉

Remote Access Services (RAS)—including protocols such as Remote Desktop Protocol (RDP), Secure Shell (SSH), Virtual Network Computing (VNC), Telnet, File Transfer Protocol (FTP), and Secure File Transfer Protocol (SFTP)—are essential to modern network infrastructures, particularly with the rise of remote work and cloud adoption. However, their exposure significantly increases the risk of brute-force attacks (BFA), where adversaries systematically guess credentials to gain unauthorized access. Traditional defenses like IP blocklisting and multifactor authentication (MFA) often struggle with scalability and adaptability to distributed attacks. This study introduces a zero-trust-aligned Software-Defined Perimeter (SDP) architecture that integrates Single Packet Authorization (SPA) for service cloaking and Connection Tracking (ConnTrack) for real-time session analysis. A Docker-based prototype was developed and tested, demonstrating no successful BFA attempts observed, latency reduction by above 10% across all evaluated RAS protocols, and the system CPU utilization reduction by 48.7% under attack conditions without impacting normal throughput. It also proved effective against connection-oriented attacks, including port scanning and distributed denial of service (DDoS) attacks. The proposed architecture offers a scalable and efficient security framework by embedding proactive defense at the authentication layer. This work advances zero-trust implementations and delivers practical, low-overhead protection for securing RAS against evolving cyber threats.

Keywords Brute-force attacks, Remote access services (RAS), Single packet authorization (SPA), Software-defined perimeter (SDP), Zero-trust architecture

Remote Access Services (RAS) have become indispensable tools for managing network resources securely from remote locations. The RAS encompasses protocols and applications such as Remote Desktop Protocol (RDP), Secure Shell (SSH), Virtual Network Computing (VNC), Telnet, File Transfer Protocol (FTP), and Secure File Transfer Protocol (SFTP). Primarily, RAS facilitates vital remote work, system administration, and collaboration within modern distributed organizations[1–3]. Due to the global transition towards remote working environments and widespread cloud adoption, the reliance on these services has dramatically increased, resulting in heightened exposure to cybersecurity threats, notably brute-force attacks (BFA)[1,2]. BFA involves attackers systematically employing automated trial-and-error techniques to compromise sensitive authentication credentials[4–8]. A simplified Fig. 1 illustrates a typical RAS system architecture and how a typical BFA works. Recent studies indicate a dramatic surge of up to 400% since 2023 in BFA campaigns targeting RAS[9], contributing significantly to ransomware incidents, data breaches, and financial losses exceeding $2.5 billion annually[10]. High-profile breaches have targeted organizations, including GitHub, Alibaba, Firefox, and the Northern Irish Parliament[11].

[1]School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China. [2]College of Information Communication and Technology, Mbeya University of Science and Technology (MUST), P.O Box 131, Mbeya, Tanzania. [3]Research and Development Department, Aifrruis Laboratories, Mbeya, Tanzania. [4]EIAS Data Science Lab, College of Computer and Information Sciences, Prince Sultan University, 11586 Riyadh, Saudi Arabia. [5]Department of Electronics and Communications Engineering, Zagazig University, Zagazig 44519, Sharqia, Egypt. [6]Center of Excellence in Quantum and Intelligent Computing, Prince Sultan University, 11586 Riyadh, Saudi Arabia. [7]Department of Mathematics and Computer Science, Faculty of Science, Menoufia University, Shibin Al Kawm 32511, Egypt. [8]Information Technology Department, College of Computer and Information Sciences, Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia. [9]Department of Computer Science, Faculty of Computers and Information, Luxor University, Luxor 85957, Egypt. ✉email: aabdellatif@psu.edu.sa; bassem.abdelatty@fci.luxor.edu.eg

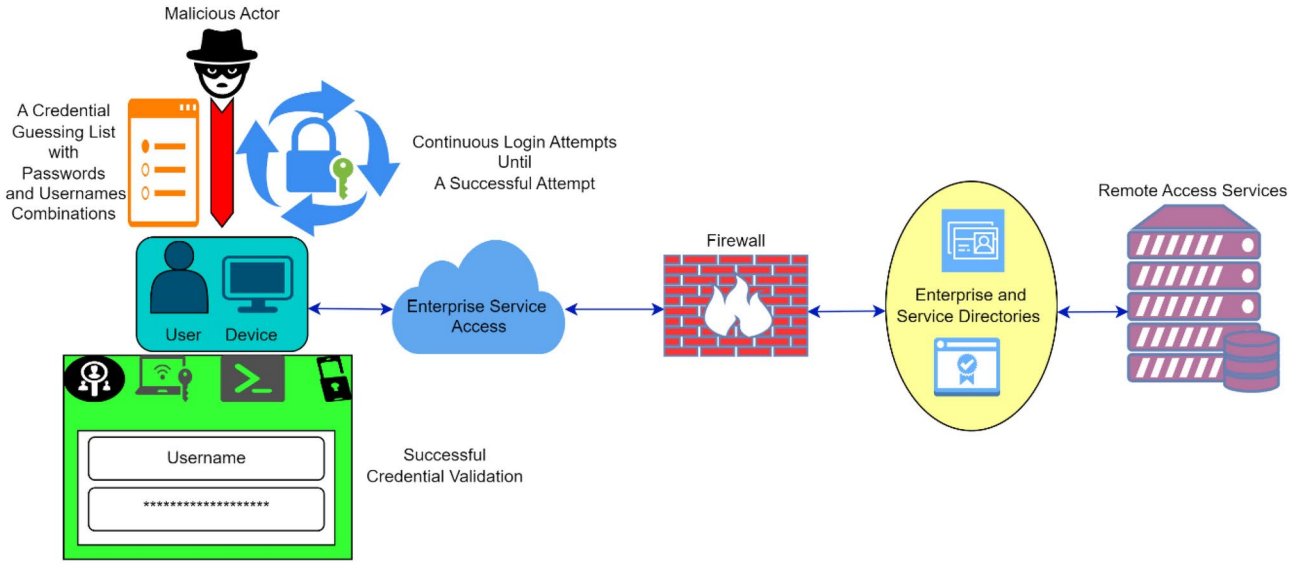


Fig. 1. Workflow of a BFA within the architecture of a RAS system.

In light of such threats' increased complexity and persistence, there is a pressing need to tighten cybersecurity protocols and governance strategies tailored to secure RAS infrastructures against evolving BFA vectors.

Existing defense strategies have important vulnerabilities. Known methods like account lockout policy can inhibit repeated guessing on individual accounts, but not if attackers randomly switch accounts or even source IP addresses. IP blocking (for example, Fail2Ban[12,13]), rate limiting[6], and multifactor authentication (MFA)[14,15] mainly use reactive tactics based on defensive action on multiple failure occurrences. Accordingly, it allows attackers to make initial attempts, enabling distributed attack evasion. Static limits have a problem with the configuration, where being too tightly configured ends up inconvenient for honest users, and being too loose grants attackers enough attempts. Although MFA strengthens security orders of magnitude, it still faces problems such as poor adoption and vulnerability to network-level attacks such as resource exhaustion attacks. These limitations highlight the need for more proactive and context-based cybersecurity strategies.

As a solid/resilient alternative to plug these significant gaps, SDP[16], built on the zero-trust security model[17], steps in. The SDP was originally put forward by the Cloud Security Alliance (CSA) in 2013; where SDP follows the principle of 'authenticate first, connect later', dynamically wrapping protected services with cryptographically enforced micro-perimeters not visible to unauthorized users[18–20]. The authenticated users authenticate with an SDP controller that dynamically establishes gateways per access rights. Unauthorized access is disregarded (silently being dropped without feedback/acknowledgment), reducing the attack surface area. Henceforth, it facilitates mitigation of port scanning, denial of service (DoS), distributed denial of service (DDoS), man-in-the-middle (MitM), and BFA.

Despite these SDP's great strengths, its convectional deployment has inherent challenges. Initially, the authentication mechanisms (for instance, SPA and mutual TLS mTLS) are cryptographically sound. Yet, they are vulnerable to consistent malicious connection attempts and can induce resource exhaustion or DoS/DDoS scenarios[21]. Furthermore, enriching SDP controllers with legacy username/password authentication for users' convenience creates vulnerabilities to credential guessing at the SDP controller layer[22–24]. On the other hand, traditional SDP realizations often do not provide sufficient and sophisticated intrusion detection and response capabilities, usually limiting the available defenses to binary allow/deny decisions, which use static, not dynamic, analyses of attacks[25,26]. As a result, persistent malicious traffic—especially insider threats or continuous attack vectors—can still create significant network noise or overload the SDP infrastructure. Lastly, while deploying SDP in legacy RAS, particularly in the setup stage, for instance, RDP, transitional attack scenarios/vulnerabilities or configuration errors are likely to happen that can expose services to BFA[27,28].

Motivated by these critical gaps, this paper introduces an Enhanced SDP Architecture optimized for proactively securing RAS against BFA. The proposed architecture uniquely integrates SPA for cryptographic invisibility of services, Connection Tracking (*ConnTrack*) for real-time session monitoring, and leveraging the Snort Intrusion Detection and Prevention System (IDPS)[29] for dynamic anomaly detection. This multi-layered approach proactively detects and mitigates credential stuffing, BFA attempts, and distributed attacks by continuously verifying authentication and rapidly responding to emerging threats. A lightweight, Docker-based SDP prototype was implemented and empirically evaluated against realistic BFA and DoS/DDoS scenarios to validate this framework. The experiments demonstrated complete mitigation of BFA targeting prevalent protocols (RDP, SSH, VNC, Telnet, FTP, and SFTP), significantly reduced resource utilization, and minimal latency impact during active threat conditions, preserving seamless performance and user experience under typical usage.

In summary, this work makes the following core contributions:

- Designing and implementing a scalable multi-layered SDP Architecture leveraging cryptographic invisibility (via SPA), real-time session monitoring (*ConnTrack*), and dynamic threat detection (Snort IDPS) prototyped as a lightweight docker container explicitly tailored for proactive BFA mitigation on RAS (across six (6) protocols (RDP, SSH, VNC, Telnet, FTP, SFTP). This architecture eliminates protocol exposure to unauthorized entities and detects brute-force attempts in real-time.
- Performance and security empirical validation of the enhanced SDP architecture against realistic BFA/DDoS scenarios on RAS (across six (6) protocols (RDP, SSH, VNC, Telnet, FTP, SFTP). To evaluate and analyze the enhanced SDP's security efficacy and resource efficiency comparatively with the conventional BFA reactive defenses on RAS.
- Providing practical insights and guidance for effective real-world implementation and adoption of zero-trust principles (via lightweight and proactive approaches such as SDP dockers templates and playbooks) in operational network environments to reduce incident response and deployability time.

The remainder of the paper is structured as follows: Section "2" presents related work summarizing previous research; Section "3" outlines the detailed design of the proposed SDP architecture and its security components; Section "4" describes the experimental setup, procedures, and comprehensive evaluations; Section "5" discusses findings, implications, and limitations; finally, Section "6" concludes the paper while highlighting the identified future research directions.

Related work

Existing brute-force attack mitigation techniques

BFA systematically attempts numerous password combinations or encryption keys to compromise user authentication, posing a significant threat to RAS. Several techniques have been proposed for mitigation, each presenting unique strengths and associated limitations as described hereafter.

Password policies, enforcing complexity and frequent updates, are commonly employed due to ease of implementation but often inconvenience users and remain vulnerable to sophisticated dictionary attacks[30–32]. Account lockout mechanisms prevent repeated attempts by temporarily deactivating accounts after multiple failures, effectively hindering BFA efforts but risking DoS scenarios and requiring significant administrative management[33–35].

MFA significantly improves security by requiring multiple credentials (for example, tamper-resistant applications[36], hardware tokens, or SMS verification). Despite reducing credential compromise risks, MFA introduces usability challenges and susceptibility to phishing and social engineering[37–40].

Network-based solutions like rate limiting and IP blocklisting/allowlisting reduce attack frequency. However, rate limiting can inconvenience legitimate users and is bypassed by distributed attacks[4,41,42]. IP-based mechanisms (for example, Fail2Ban) effectively block known malicious addresses but are limited against IP spoofing, dynamically allocated addresses, and distributed botnet attacks, creating extensive administrative overhead[43,44]

Recent advancements leverage machine learning (ML) and deep learning (DL), analyzing network telemetry[7,8,45,46], or behavioral biometrics such as typing patterns[7,47]. These methods offer adaptive attack detection capabilities but entail significant data processing overhead and raise privacy concerns due to extensive user behavior monitoring[39,48,49]. Table 1 summarizes the advantages and limitations of these techniques.

Addressing RAS security gaps with software-defined perimeter (SDP) architecture

Traditional BFA mitigation strategies motivate adopting SDP, a zero-trust security architecture that dynamically enforces context-aware access controls, continuously authenticating users, devices, and applications[16,17,50]. Unlike traditional static approaches, SDP minimizes the attack surface through micro-segmentation, isolating critical resources within secure micro-perimeters and enabling granular security enforcement and dynamic threat adaptation[51,52]

SDP integrates seamlessly with MFA, complementing and enhancing protection by continuously verifying multiple authentication factors and monitoring session contexts, thereby mitigating risks associated with compromised credentials and phishing[17,23,24]. Prior studies validated SDP's efficacy against volumetric attacks, demonstrating improved resilience to DDoS and port-scanning while incurring modest latency overhead (approximately an average of 1s connection setup)[18,19,53].

However, prior research has not explicitly integrated dedicated BFA mitigation within SDP's authentication layer. Our research bridges this gap by embedding proactive BFA detection directly within SDP, enabling adaptive threat response and minimizing exposure to attackers. Moreover, formal verification of SDP resilience

Technique	Advantages	Limitations
Password policies	Simple to implement	User inconvenience, vulnerable to sophisticated attacks[32]
Account lockout	Prevents continuous attempts	Risk of DoS, management overhead[34,35]
Multifactor authentication	High-security level	Additional user steps, potential phishing risks[37–40]
Rate limiting	Reduces attack speed	Circumvented by distributed attacks, it affects legitimate users[4,41,42]
IP blocklisting/Allowlisting	Effective against known IPs	Limited against spoofing, dynamic IPs, management overhead[43,44]
Behavioral biometrics	Non-intrusive, challenging to mimic	Data collection and privacy concerns[39,48,49]

Table 1. Advantages and limitations of BFA mitigation techniques.

against BFA remains unexplored; thus, our contribution extends zero-trust principles to active authentication-layer intrusion prevention and real-time threat adaptation. This study addresses critical limitations in existing approaches, delivering robust, proactive SDP-based protection for RAS against evolving BFA.

Proposed software-defined perimeter architecture

SDP system architecture

In this work, the design and implementation of the SDP system architecture for mitigating BFA on RAS starts by identifying prominent RAS assets and their risk level. Then, understanding their operation anatomy through identifying and recording each RAS asset's connections, including sources, destinations, protocols, and ports. To this end, selecting an appropriate SDP architecture that fits the number of RAS assets and the required security level is critical. Zero-trust security policies for each RAS asset can be established based on the tailor-made SDP architecture for RAS, specifying which connections are allowed or denied based on strict authentication and authorization rules. Lastly, continuously monitor system events for signs of BFA attempts, log these events, and promptly address any detected issues while reassessing each RAS asset's criticality and updating the flow mappings to maintain adequate protection against evolving threats.

Figure 2 illustrates the SDP architecture's primary components for securing RAS. The primary architecture for essential SDP operations comprises the system components, including the SDP Client Initiating Host (SCIH), SDP Service Accepting Host (SSAH), SDP Gateway (SG), and SDP Controller (SC). The SCIH, i.e., the user and device, and SSAH (protected resource/application) are connected to SC, the authentication center, and the policy engine.

The SC assigns the appropriate policy to the SCIH, determining/resolving its allowed resource access. The SC provides the control layer and enables authentication of SCIH and SSAH before allowing any communication. The SPA protects the SC and SSAH by limiting their accessibility and visibility to unauthorized network entities, i.e., users, devices, and applications. As a result, every network entity is cloaked/blackened/hidden from users. The SG sits before the resources to protect them, allowing only authorized network entities, i.e., devices, users, and applications, to access the secured services and processes. Once the SC grants the policy to the CISH, the SG opens the specific port to allow only the specified service and blocks any other communication, i.e., it is a default-deny-all firewall. Figure 3 shows the SDP architecture's connection setup. It details how SC connects to services for authentication and authorization and how SG manages secure communications with encrypted connections and enhanced authentication.

The SDP services access, Hash-based Message Authentication Code (HMAC), and encryption keys management operations involve several steps and processes. The main objective is establishing a secure tunnel to access services only to legitimate clients, i.e., verified and assigned dynamic, least-privilege, and fine-granular policies, via the SDP Gateway. The established tunnel between the legitimate client and the SDP gateway resets service access by closing the firewall, i.e., removing all rules and returning to the default, denying all settings every 30 s. As shown by the flowchart in Fig. 4, after the introduction and initiation of the SDP primary components, i.e., the SDP controller, the SDP gateway, and the client, the operation starts with the client application sending an HTTP request every 25 s to the SDP controller to get HMAC and encryption keys. Thus, this operation allows the legitimate client to have continuous access to the requested services upon successful connection with the gateway.

When the SDP controller receives the HTTP request from the legitimate client, the *fwknop* tool generates the HMAC and encryption keys incorporating multiple attributes, including a unique random number (such as timestamps and nonce), protocols, port numbers, and device identity. The SDP controller sends the generated

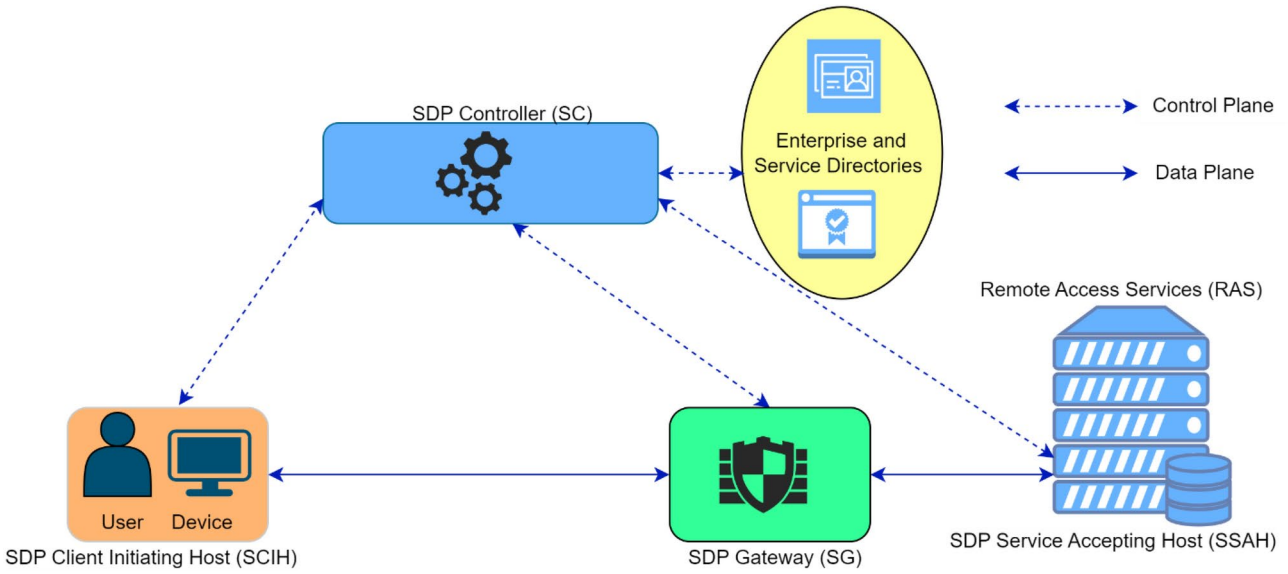


Fig. 2. The SDP architecture and the primary system components.

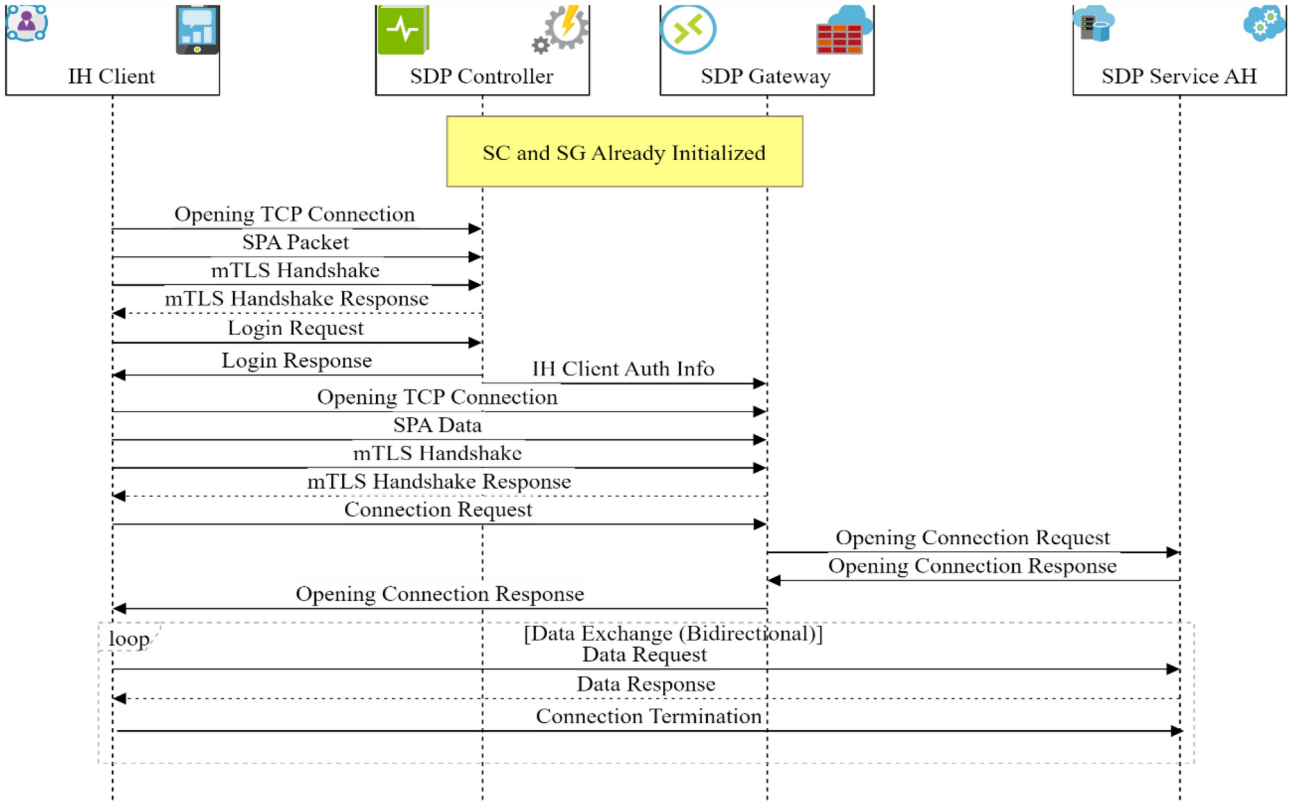


Fig. 3. SDP architecture connection setup, showing SC service connections for authentication and authorization and SG management of secure, encrypted communications with enhanced authentication.

HMAC and encryption keys to the gateway server before responding to the client. The gateway server receives the generated keys for the user and updates the respective stanza file for successful key verification at a later stage. After updating the gateway, the client who initiated the access request receives the updated HMAC and encryption keys from the SDP controller and updates its local *fwknoprc* stanza file, which matches the updated gateway's fwknopd stanza file. Then, the client sends a *fwknop* knock request to the gateway to access services. The *fwknopd* does not reply; if the keys match, access is given; otherwise, the client lacks access. This repetitive and ongoing process allows services access to only legitimate entities. Algorithm 1 also depicts the SDP access, HMAC, and encryption key management operations.

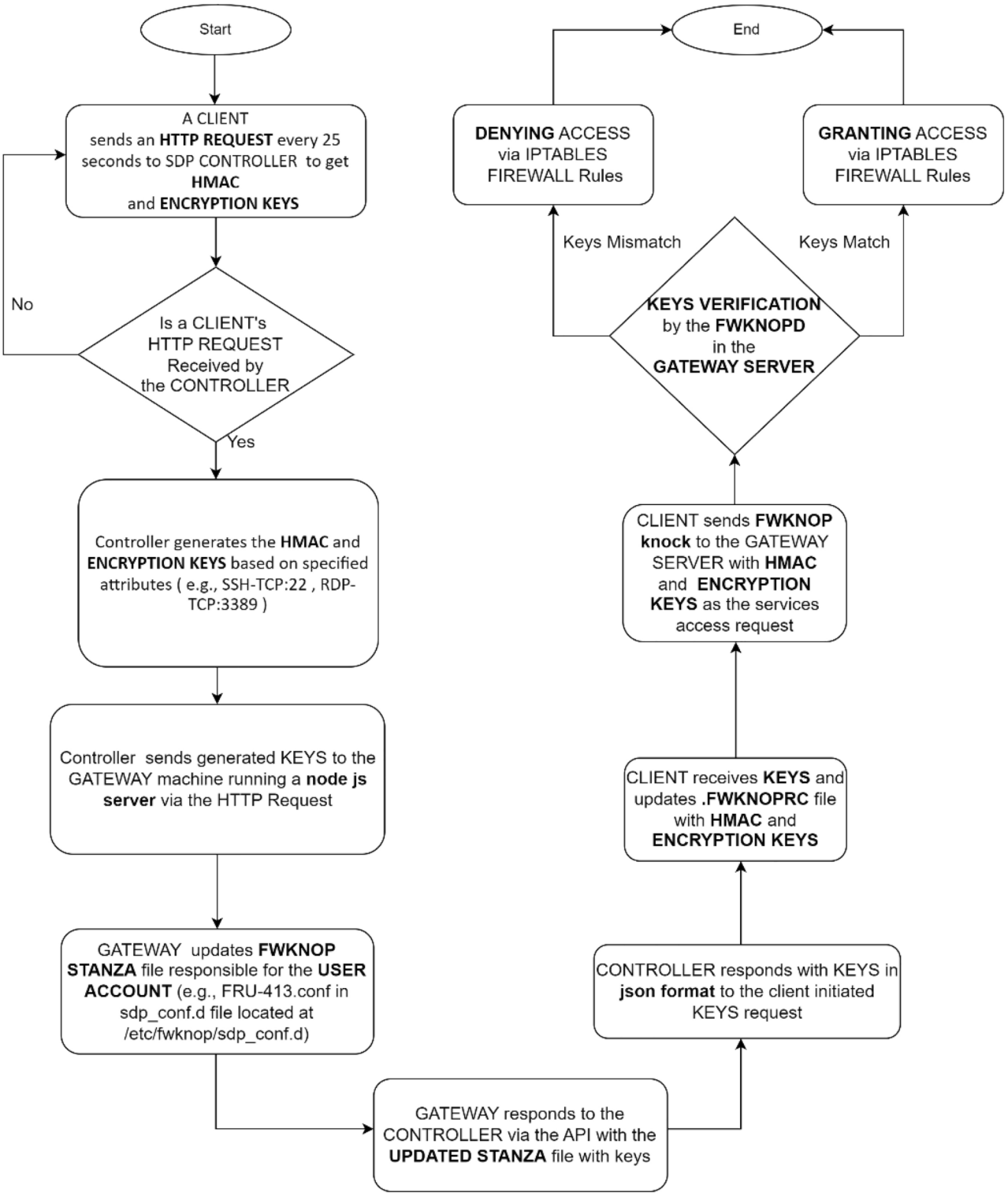


Fig. 4. A flowchart for HMAC and encryption keys management operation for continuously securing RAS in the proposed SDP architecture.

1. *Client request HMAC and ENCRYPTION KEYS to SDP from the controller after every predefined time interval.*
2. **While** (*is_request_received( )== false*) **Do**
3. *Go to step 1*
4. **End While**
5. *Controller generates the HMAC and Encryption Keys*
6. ***Do*** *Send Generated KEYS to the GATEWAY*
7. *Gateway update FWKNOP STANZA file*
8. **End While** *(is_file_updated( ) == False)*
9. *Send updated stanza file with keys to the controller*
10. *Controller responds with keys to the client-initiated key request*
11. *Client update.FWKNOPRC file with keys received*
12. *Client sends FWKNOP knock to the Gateway server*
13. **If** *is_FWKNOP_knock_success ( ) == True* **Then**
14. *Grant access via IPTables and Firewall rules*
15. **Else**
16. *Go to step 1*
17. **End If**

Algorithm 1. SDP Services Access and Maintenance Operation

1. Features and Techniques

SPA: SPA is a technique used within SDP architecture to enhance security by ensuring that network services are hidden and inaccessible to unauthorized users. SPA requires an authorized user to send a specially crafted packet (authorization packet) to the SDP gateway. This packet serves as a "knock" to reveal the hidden service.

ConnTrack: *ConnTrack* is a feature in network firewalls that tracks the state of network connections. It maintains a state table of active connections, allowing the firewall to decide based on the traffic state.

Snort IDS: Network Intrusion Detection & Prevention System (Snort) is an open-source Intrusion Detection and Prevention System (IDS/IPS) that monitors network traffic for suspicious activity. In our setup, Snort IDS/IPS is integrated into the SDP architecture for validation and extensibility. Snort provides an extra security layer by monitoring traffic and identifying intrusions that bypass SDP, helping us continuously improve our security measures. Additionally, Snort demonstrates the flexibility of our SDP architecture, showing how it can work seamlessly with existing security tools. Combining SDP and Snort creates a more robust, layered defense against threats, enhancing traditional security methods.

2. Effectiveness of SPA and ConnTrack Synergy in Mitigating BFA and Associated Threat Vectors on RAS

Hiding RAS: By hiding services behind the SDP gateway and only revealing them to authorized users through SPA, BFA attackers cannot even detect the existence of the RAS to initiate attacks.

Limited Exposure: SPA ensures that only authenticated requests reach the SDP gateway, significantly reducing the attack surface and making it difficult for attackers to perform BFA on RAS.

Reduced Noise: SPA helps filter out unauthorized traffic, reducing the chances of associated threat vectors such as DoS/DDoS attacks overwhelming the gateway and protecting RAS resources.

Stateful Inspection: *ConnTrack* performs stateful inspection of packets, ensuring that only legitimate, stateful connections are allowed through the SDP gateway. This functionality/capability helps block unauthorized connection attempts and mitigate BFA on RAS.

Rate Limiting: By monitoring and managing connection states, *ConnTrack* can enforce rate limiting on connection attempts, making it difficult for attackers to perform rapid, successive connection attempts typical in BFA and DoS/DDoS attacks on RAS.

Anomaly Detection: *ConnTrack* can detect anomalies in network traffic patterns, such as unusual spikes in connection attempts, indicative of DoS/DDoS attacks. It can then trigger appropriate countermeasures to protect RAS and the network.

SPA and *ConnTrack*, when integrated into the SDP architecture, provide a robust mechanism for securing RAS against BFA and DoS/DDoS attacks. SPA effectively hides services from unauthorized users, while *ConnTrack* ensures that only legitimate connections are maintained and monitored for anomalies. Synergetically, SPA and *ConnTrack* create a secure, dynamic, and resilient perimeter that adapts to evolving threats, providing enhanced security for RAS.

Formal system model

To rigorously assess the security of the enhanced SDP architecture, we define a formal system model capturing essential entities, states, protocols, and attacker capabilities, enabling formal verification of critical security properties. The system is represented as a tuple:

$$S = \{C, AS, PE, G, U, D, R, P, BF, IDS, M, A, SDP\}$$

where

1. Sets

$C = \{c_1, c_2, \ldots c_x\}$: Each c_i is an SDP controller for managing authentication.
$AS = \{as_1, as_2, \ldots as_y\}$: Each as_i is an authentication server verifying credentials.
$PE = \{p_{e1}, p_{e2}, \ldots p_{ez}\}$: Each p_{ei} is a policy engine evaluating access rules.
$G = \{g_1, g_2, \ldots g_w\}$: Each g_i is an SDP gateway enforcing secure access.
$U = \{u_1, u_2, \ldots u_n\}$: Each u_i is a user attempting access.
$D = \{d_1, d_2, \ldots d_m\}$: Each d_i is a user device.
$R = \{r_1, r_2, \ldots r_l\}$: Each r_i is a network resource protected by SDP.
$P = \{p_1, p_2, \ldots p_k\}$: Each p_i *is* an access policy defining authorization rules.
$BF = \{bf_1, bf_2, \ldots bf_0\}$: Each bf_i is a BFA event.
$IDS = \{ids_1, ids_2, \ldots ids_v\}$: Each ids_i is a Snort IDS/IPS instance.
$M = \{rxCount, txCount, cpuPercent, latency_p, throughput_p, packetLoss_p | p \in P\}$: Set of network metrics, where the subscript *p* indicates the RAS protocol for which the metric is measured.
$A \in \{0, 1\}$: Attack state indicator (0: no attack, 1: attack active)
T: Set of discrete time points.

2. Functions

The system evolves through clearly defined dynamics, including authentication, authorization, connection establishment, and security event detection and mitigation:

Authentication: A user is considered authenticated on a device at a given time if and only if both the user's credentials and the device's certificate are successfully verified by an authentication server, along with a nonce exchange to prevent replay attacks:

$Auth\,(u, d, t) \in \{0, 1\}$: 1 if a user "*u*" is authenticated on device "*d*" at time "*t*," 0 otherwise.

$$\text{Auth}\,(u, d, t) = 1 \Leftrightarrow \left(U \overset{\text{Credentials,Nonce}}{\rightarrow} AS\right) \wedge \left(D \overset{\text{Certificate,Nonce}}{\rightarrow} AS\right)$$

Authorization: A user is authorized to access a resource from a device at a given time if and only if the user is authenticated on the device and the policy evaluation for the access request is successful:

$AuthZ(u, d, r, t) \in \{0,1\}$: 1 if user *"u"* is authorized to access resource *"r"* from device *"d"* at a time *"t,"* 0 otherwise.

Where, $PolicyEval(u, d, r, P, t) \in \{0,1\}$: 1 if the policy evaluation of user's *"u"* request from device *"d"* to access resource *"r"* under policies *"P"* a time *"t,"* is successful, *"0"* otherwise.

$$\text{Auth } Z\,(u, d, r, t) = 1 \Leftrightarrow (\text{Auth}\,(u, d, t) = 1) \wedge (\text{PolicyEval}\,(u, d, r, P, t) = 1)$$

Connection Establishment: A device is considered connected to a gateway at a given time if and only if the user is authenticated on the device and a secure tunnel is successfully established between the device and the gateway:

$Conn(d, g, t) \in \{0,1\}$: 1 if device *"d"* is connected to gateway *"g"* at a time *"t,"* 0 otherwise.

Where, $EstablishTunnel(d, g, t) \in \{0,1\}$: 1 if a secure tunnel is established between device *"d"* and gateway *"g"* at a time *"t,"* 0 otherwise.

$$\text{Conn}\,(d, g, t) = 1 \Leftrightarrow (\text{Auth}\,(u, d, t) = 1) \wedge (\ \text{EstablishTunnel}\ \ (d, g, t) = 1)$$

BFA Detection and Mitigation: $DetectBruteForce(u, d, t) \in \{0,1\}$: 1 if a BFA by user *"u"* on device *"d"* is detected at a time *"t,"* 0 otherwise.

$$\text{MitigateBFA}\ \ (u, d, t)\ \text{ is triggered if DetectBFA}\ \ (u, d, t) = 1$$

Intrusion Detection: The $DetectIntrusion(ids, t)$ function is evaluated at each time *"t,"* for each Snort IDS/IPS instance $ids \in IDS$. If an intrusion is detected $DetectIntrusion\,(ids, t) = 1$, an alert is generated and sent to the SDP Controller.

$DetectIntrusion(ids, t) \in \{0,1\}$: 1 if an intrusion is detected by Snort instance *ids* at a time *"t,"* 0 otherwise.

3. Quantitative Security Analysis

This subsection presents a security analysis that uses mathematical models and statistical methods to quantify security properties and evaluate the effectiveness of security mechanisms. The critical elements of the proposed

SDP architecture under this quantitative security analysis are Mutual Authentication Security, SPA Security, and Dynamic Firewall Effectiveness.

Mutual Authentication Security: The probability of a successful authentication attempt given a BFA can be modeled as:

$$P(\text{Auth}(u, d, t) = 1 \mid bf \in BF) = \epsilon$$

where ϵ is a small probability representing the chance of the attacker guessing the correct *credentials* and *nonce*.

SPA Security: The probability of successful unauthorized access to a resource using an intercepted token is:

$$P\left(\text{Auth}Z\left(u, d, r, t\right) = 1 | \text{Auth}\left(u, d, t\right) = 0\right) = \delta$$

where δ is a small probability due to the short-lived and resource-bound nature of the tokens.

Dynamic Firewall Effectiveness: The effectiveness of dynamic firewall rules in mitigating attacks can be quantified by comparing the average number of successful attacks with and without SDP being enabled:

$$E_{\text{firewall}} = \frac{E\left[\sum_{t\in T} X_{t,p,\ \text{packetLoss}} \mid \text{SDP} = 0, A = 1\right] - E\left[\sum_{t\in T} X_{t,p,\ \text{packetLoss}} \mid \text{SDP} = 1, A = 1\right]}{E\left[\sum_{t\in T} X_{t,p,\ \text{packetLoss}} \mid \text{SDP} = 0, A = 1\right]}$$

where

- SDP = 0 indicates SDP is disabled.
- SDP = 1 indicates SDP is enabled.
- A = 1 indicates the scenario is under attack conditions.

4. Algorithms

The algorithms designed to mitigate BFA and network intrusion attacks using the SDP architecture in the context of RAS combine authentication, authorization, connection establishment, and security monitoring to enhance network security. Specifically, after initialization, load system configurations. Initialize authentication servers (AS), policy engines (PE), SDP controllers (C), and Snort IDS/IPS instances (IDS). Algorithms 2, 3, 4, 5, and 6 provide a high-level overview of the earlier described SDP architecture's operation tailored to fulfill the authentication and authorization, connection establishment, and security monitoring and mitigation, respectively.

Algorithm 2: This procedure handles user authentication and authorization on a device trying to access a respective RAS resource.

Input: *User credentials (U), device certificate (D), nonce (N)*
Output: *Authentication status (Auth), authorization status (AuthZ)*

```
Procedure AuthenticateAndAuthorize(U, D, N, r, P)
  Auth(U, D, t) = AS.VerifyCredentials(U, N) AND AS.VerifyCertificate(D, N)
  If (Auth(U, D, t))
    AuthZ(U, D, r, t) = PE.EvaluatePolicy(U, D, r, P, t)
  End If
  Return Auth, AuthZ
```

Algorithm 2. Authentication and Authorization

Algorithm 3: This procedure establishes a secure tunnel between a device and the SDP gateway upon successful authentication and updates the *ConnTrack* table.

Input: *Authentication status (Auth), device (D), gateway (G)*
Output: *Connection status (Conn)*

```
Procedure EstablishConnection(Auth, D, G)
  If (Auth)
    Conn(D, G, t) = EstablishTunnel(D, G, t)
  End If
  Return Conn
```

Algorithm 3. Connection Establishment

Algorithm 4: This function detects BFA based on the number of failed login attempts within a time window.

Input: *User credentials (U), device certificate (D), time (t)*
Output: *BFA detected (1) or not detected (0)*

```
failed_attempts ← GetFailedAttempts(U, D, t) // Retrieve the number of failed authentication attempts within a defined time window for the given user and device.
If failed_attempts_count > THRESHOLD then
    Return 1  // BFA detected
Else
    Return 0  // No BFA detected
End If
```

Algorithm 4. BFA Detection

Algorithm 5: This function is implemented within the Snort IDS/IPS and analyzes network traffic.

Input: *Snort IDS/IPS instance (ids), time (t)*
Output: *Intrusion status (is_intrusion_detected)*

```
traffic_data ← CaptureNetworkTraffic()
is_intrusion_detected ← 0  // Initialize as no intrusion
  For each rule in ids.Ruleset do
    If rule.Matches(traffic_data) then
      Return 1  // Intrusion detected
      Break     // Exit loop early if intrusion found
    End If
  End For
  Return is_intrusion_detected
```

Algorithm 5. Intrusion Detection

Algorithm 6: This procedure encapsulates the main loop of the SDP architecture, continuously monitoring and responding to security events.

```
InitializeSystem()
While (system is running) do
  For each user u, device d, resource ras, do
    auth_success, authz_success← AuthenticateAndAuthorize(u, d, GenerateNonce(), ras, P)
    If (authz_success) then
      conn_success ← EstablishConnection(auth_success, d, G)
      If (conn_success), then
        CT.AddConnection(u, d, ras, t)  // Add connection to ConnTrack table
      End If
    End If
    If (Detect BFA(u, d, t)) then
      MitigateBFA(u, d, t)
    End If
  End For
  For each IDS instance, "ids"  in IDS do
    If (DetectIntrusion(ids, t)) then
      C.ProcessAlert(ids)
    End If
  End For
  For each protocol "p,"  and metric "m," in M do
    CollectMetrics(p, m, t)
  End For
  CT.UpdateConnections()  // Update ConnTrack table (remove expired connections, etc.)
End While
```

Algorithm 6. Security Monitoring and Mitigation

Experimental evaluation

Experimental setup

The proposed enhanced SDP architecture was implemented and evaluated using a containerized test environment deployed on a physical host machine running Ubuntu Linux 20.04 LTS with Docker Engine. This setup consists of four Docker containers, each simulating distinct roles within the architecture to facilitate performance and security evaluation under realistic attack scenarios.

The *SDP Controller* component was deployed in *Container 1*, running a custom implementation of the Waverley SDP controller module. *Container 2* served as the *SDP Gateway,* implementing a deny-all firewall policy using *iptables* and a customized configuration of the fwknop module to represent the Accepting Host (AH). This gateway was responsible for cloaking protected RAS and managing access enforcement. The *attacker environment* was emulated in *Container 3* using Kali Linux, equipped with *BFA* scripts to simulate unauthorized access attempts on the RAS protected by the proposed system. Lastly, *Container 4* acted as a *legitimate client,* configured with the Initiating Host (IH) version of the fwknop module to generate valid access requests.

Component	Software stack	Hardware allocation	Role description
Host Machine	Ubuntu Linux 20.04 LTS, Docker Engine	Intel Core i7-7700HQ @ 2.80 GHz (8 MB Cache), 16 GB DDR4 RAM, NVIDIA GTX 1050 4 GB GDDR5, 1.0 Gbps Ethernet	Host platform for all containers
Container 1	Ubuntu Linux 20.04 LTS, Waverley SDP Controller Module	1 vCPU, 2 GB RAM, 2 NICs	Runs SDP Controller
Container 2	Ubuntu Linux 20.04 LTS, fwknop (AH configuration), RAS Services	1 vCPU, 2 GB RAM, 3 NICs	Acts as SDP Gateway with iptables firewall
Container 3	Kali Linux, Attack Scripts	1 vCPU, 1 GB RAM, 1 NIC	Simulates BFA and DoS attack scenarios
Container 4	Ubuntu Linux 20.04 LTS, fwknop (IH configuration)	1 vCPU, 1 GB RAM, 1 NIC	Emulates legitimate client using SPA authentication

Table 2. Experimental testbed specifications and descriptions.

All containers were deployed on a single physical host machine with sufficient hardware resources to ensure realistic emulation of network operations. Table 2 provides the detailed specifications of the host machine and each container, including allocated hardware resources, operating systems, and component roles.

The implemented SDP prototype comprises multiple components integrated to achieve the main objective of the system development. We have incorporated usability and other related metrics to meet the intended tasks. The SDP infrastructure implementation setup in this experimentation is the customized version of the open-source SDP architecture prototype from the Waverly Labs[54]. It is also available in a dockerized setup via Aifrruis Labs[55,56]. The infrastructure comprises the SDP controller, which manages clients' identities and trust score computation (Based on context-aware metrics such as username and credential, MAC address, Geolocation, and Face recognition) for client authentication and verification.

The developed SDP controller technologies are *Vue JS* for the front end, *Laravel* for the backend, i.e., API, *swagger* for API documentation, and *MySQL* version *8.0.27* for the database. Also, the SDP infrastructure possesses an SDP gateway with allocated services operating with the deny-all policy by default, which accepts access only to authenticated and verified clients by the SDP controller. The port mappings via the iptable tool facilitate the filtering and routing for only allowed services as per predefined business policies with the connection status tracking capability. Table 3 shows the port mappings for the service configurations in this SDP prototype implementation. The desktop client is another component of the implemented SDP infrastructure, the lightweight software application facilitating authentication, verification, and maintaining client access to the protected gateway services. The detailed SDP controller interface and the desktop client of the developed SDP prototype framework are shown in the Appendix and Figs. 14 and 15 are shown, respectively.

Experimental procedure

The experimentation procedure involves three phases: phase one validates and demonstrates the efficacy of the proposed SDP framework against BFA and DDoS attacks targeting critical RAS—namely RDP, SSH, VNC, Telnet, FTP, and SFTP. The testing employs widely recognized cybersecurity tools: *THC Hydra v9.0*[57]for automating BFA via credential guessing, *Nmap v7.80*[58] for service discovery via port scanning, and *hping3* for realistic DDoS simulation. These specific tools were selected due to their widespread adoption, versatility, and realism, enabling accurate emulation of genuine attacker behaviors[6,7,59,60]. Initially, *Nmap* scans were conducted multiple times to identify available RAS services within two experimental scenarios—SDP-enabled and SDP-disabled architectures. Following this reconnaissance phase, *THC Hydra* was utilized to execute BFA on the detected RAS endpoints systematically. Each BFA is terminated upon successful credential validation or after reaching a predefined maximum threshold of login attempts. Simultaneously, *hping3* simulated realistic network-layer DDoS attacks to assess the SDP framework's resilience further. Figure 5 illustrates the flowchart depicting the detailed procedure of these BFA tests.

In phase two, we conducted a performance evaluation on connection setup, latency, throughput, packet loss, and CPU usage as the performance metrics upon comparison with the proposed SDP framework performance against the convectional RAS setup, i.e., when SDP is disabled. We measure the connection setup time for the developed SDP architecture prototype when a client with permission connects to the SDP gateway by using "Postman"(a widely-used API development and testing tool that simplifies the process of creating, sending, and managing HTTP requests to APIs)to send a request to the SDP gateway. After receiving the response, the "Timings" section in the "Tests" tab was checked, revealing details like "DNS Lookup," "Initial Connection," "SSL Handshake," and "Time to First Byte," which were used to assess the connection setup time. Regarding the throughput, we use the Iperf tool to measure the throughput between the SDP gateway enclaving RAS and the client and record the results. Tools like "ping" (for latency and packet loss) and "netcat" (for real-time latency) were used to understand network response times and data loss. Also, via the devised prototype's SDP controller interface, we recorded and gauged real-time network traffic and CPU usage statistics collected from the system hardware in both scenarios when SDP is enabled and when it is disabled with attacks and without attacks for the RAS. These measurements were repeated several times, and additional diagnostic tools like "my traceroute" (MTR) were used to understand better the SDP gateway's performance and the whole SDP architecture under similar scenarios. Through this thorough approach, we ensured a comprehensive and reliable analysis.

In phase three, we conducted a comparative performance and security analysis of the enhanced SDP architecture versus the existing BFA mitigation approaches. We conducted a comparative study by iteratively

Remote access service type	Port inside docker	Port outside docker
HTTP, Apache2, Phpmyadmin, and Related Services	80	11800
HTTPS	443	11801
SSH and SFTP	22	11802
FTP	21	11803
Mysql Service	3306	11804
Iperf	5001	11805
Telnet	23	11806
RDP	3389	11809
VNC	5901	11810

Table 3. Ports mapping for services running on dockerized implemented SDP architecture prototype.

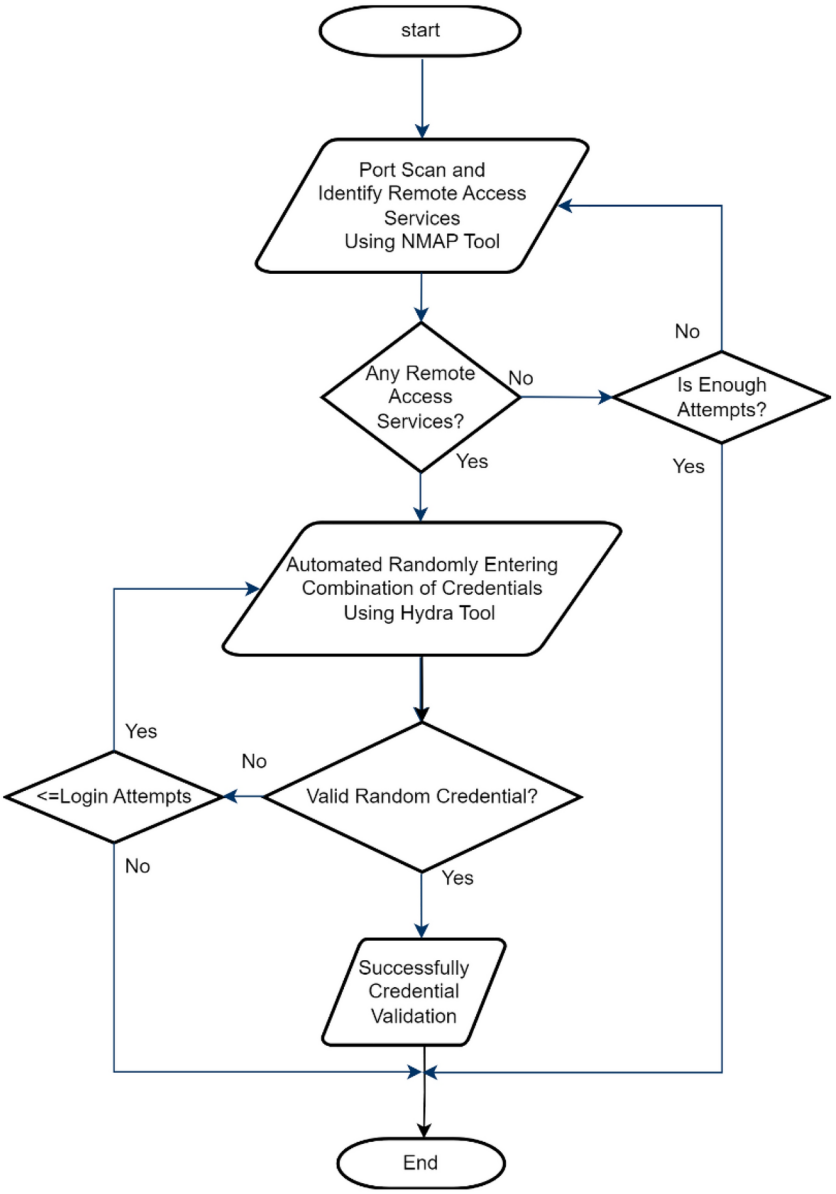


Fig. 5. A flowchart of a typical BFA operation.

integrating conventional security methods for BFA mitigation: VPN, MFA, ML, Rate Limiting, and Fail2Ban. Towards this comparative performance and security analysis study, the VPN approach leveraged OpenVPN's open-source solution to provide encrypted and secure communication channels to obscure endpoints from brute-force attacks and network reconnaissance. A robust multi-layered verification was implemented for MFA, combining device identification and username-password credentials. The rationale was to assess the effectiveness of strong identity validation layers against credential-based attacks. The ML approach was implemented via selected pre-trained models within SnortML, enabling adaptive anomaly detection based on historical traffic patterns to identify and mitigate brute-force attempts proactively.

Additionally, rate limiting and Fail2Ban methods were implemented via customized SnortML rules that monitor login attempt frequencies and automate IP blocking upon exceeding predefined thresholds, thereby reducing attack surface exposure. This unified integration provided a clear comparative perspective on each method's effectiveness, efficiency, and practical overhead relative to the proposed SDP architecture.

Security and performance analysis

Security analysis

Table 4 summarizes the Nmap port scanning and BFA results in the experimental setup with SDP architecture enabled and disabled. With the SDP-enabled architecture, the targeted RAS, i.e., RDP, VNC, SSH, Telnet, SFTP, and FTP, are hidden behind the SDP gateway with the default-deny all firewall, making them invisible by unverified users and applications. Hence, the Nmap tool unsuccessfully discovers the protected RAS during

Attacking tool	Remote access service	With SDP disabled	With SDP enabled
Nmap for port scanning	RDP, VNC, SSH, Telnet, SFTP and FTP	Discoverable as Open ports 3389, 5901, 21, 23, 21, and 22, respectively	All ports are filtered and not discoverable. RAS is cloaked
Hydra for BFA	RDP, VNC, SSH, Telnet, SFTP and FTP	Susceptible to BFA	RAS never accepts requests from unverified users/applications

Table 4. Security analysis results of RAS against port scanning and BFA.

SDP status	With SDP disabled	With SDP enabled
Time (in seconds)	0.12	1.05

Table 5. Time for connection setup.

port scanning. Conversely, with the SDP-disabled architecture, RAS and related services are discoverable and thus susceptible to port scanning via the Nmap tool, leading to BFA. Figure 16a and b (in the Appendix) clearly illustrate the Nmap tool's port scanning outcomes with the SDP architecture in its enabled and disabled states, respectively.

Likewise, the RAS within the SDP-enabled architecture, RDP, VNC, SSH, Telnet, SFTP, and FTP, for example, are intrinsically secure against BFA. Since services in the SDP-enabled architecture are provisioned and accessed based on a need-to-know fashion and in the least privileged fashion after thorough verification, only verified users and applications can send requests and access them upon qualifying access resource policies. In contrast, the RASs within the SDP-disabled architecture are susceptible to BFA. They allow trial and error attempts for login and credentials before verifying (following the connecting first, then verifying the next paradigm). For a detailed demonstration, see Appendix, Figs. 17–22 (denoted with "a" for the SDP-enabled architecture and "b" for the SDP-disabled architecture) illustrate the BFA results conducted against RDP, VNC, SSH, Telnet, SFTP, and FTP, respectively.

Performance analysis

1. Connection Setup Time

In SDP-enabled environments, the SDP components experience trivial startup time overhead compared to the environments without the application of the SDP. The overhead is primarily due to two additional steps: a client has to send encryption and HMAC requests, and a server has to send a request to the gateway to update Stanza. Specifically, the initiating host authentication takes 595 ms for encryption and HMAC requests. Also, the accepting host authentication time incurs an extra 455 ms for a server to request the gateway to update Stanza. According to our experimentation setup, the aforementioned additional steps are not of concern within the regular network infrastructure, which needs only a 0.12-ms connection setup, as shown in Table 5. Similarly, previous research works[18,53], and[60] reported such overhead, which varies in size based on the system components' computation capability and the link speed between participated entities/components.

2. Network Throughput

The recorded throughput in our experiment justifies that the proposed SDP architecture deployment does not significantly affect the infrastructure throughput. It does not affect either peer's connection within the infrastructure. Instead, the bandwidth bottlenecks are the network interfaces, connection links, and processing capabilities of the connected peers. For a real depiction of this, see Appendix, Fig. 23a and b, with SDP enabled and disabled, respectively.

However, across RAS protocols, the throughput performance evaluation with the proposed SDP architecture shows significant changes from the setup without SDP architecture. The throughput is significantly improved when the SDP architecture is enabled during the attack scenarios across the RAS protocols. The boxplots in Fig. 6 illustrate the Mbps throughput when the SDP is enabled and disabled for all four scenarios: under normal (without attacks) conditions and under attack conditions. On the one hand, with the SDP-disabled mode under normal conditions, the RDP throughput has a median of 10.19 Mbps (Mean = 10.15 Mbps, Standard deviation (STD) = 1.94 Mbps, and Variance = 3.78 Mbps). On the other hand, with the SDP-enabled mode under normal conditions, the RDP throughput had a median of 9.89 Mbps (Mean = 10.03 Mbps, Standard deviation (STD) = 2.20 Mbps, and Variance = 4.83 Mbps). Meanwhile, SDP generally improves throughput during attacks, especially for RDP, SSH, VNC, and FTP. For precise clarification via statistical significance confirmation, the descriptive statistical information of the gateway throughput results for respective RAS protocol under the four scenarios is given in the Appendix section by Fig. 24a (Normal with SDP disabled mode), b (Normal with SDP enabled mode), c (under attack with SDP disabled mode), and d (under attack with SDP enabled mode). During attacks, SDP maintained 4.0 times higher RDP throughput (7.91 Mbps vs. 1.99 Mbps without SDP) and 3.1 times higher FTP throughput (9.90 Mbps vs. 3.10 Mbps).

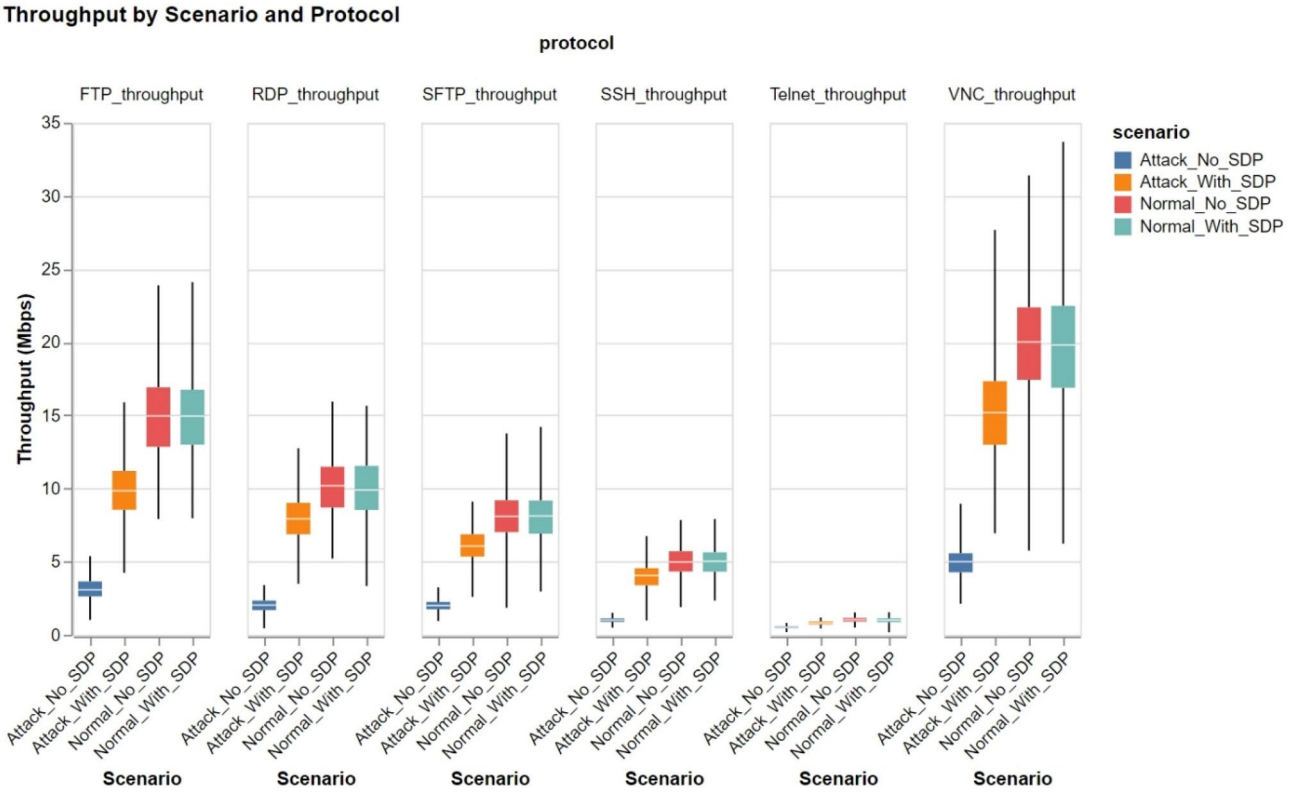


Fig. 6. Measured throughput for all RAS protocols when the SDP is enabled and disabled under attack and normal conditions.

3. Latency

The performance evaluation demonstrates that the proposed SDP architecture significantly reduces latency across all RAS protocols. Under normal conditions, RDP latency decreased by 11% (median: 50.09–45.02 ms), attributed to SDP's optimization of UDP communication and elimination of ad-hoc security checks. During attacks, SDP achieved even greater reductions: RDP latency improved by 48.2% (median: 194.39–100.76 ms; 95% CI [98.64, 102.89 ms]), and VNC by 50.6% (median: 297.48–147.33 ms). Also, FTP latency decreased by 22% (median: 248.11–125.45 ms). These improvements correlate with reduced variance (RDP's standard deviation dropped 22.3%), indicating more predictable network conditions. For precise clarification via statistical significance confirmation, the descriptive statistical information of the gateway latency results is given in the Appendix section by Fig. 24a (Normal with SDP disabled mode), b (Normal with SDP enabled mode), c (under attack with SDP disabled mode), and d (under attack with SDP enabled mode). Under Attack: SDP reduced median latency by 48.2% for RDP (194.39–100.76 ms) and 50.6% for VNC (297.48–147.33 ms) (Fig. 7).

4. Packet Loss

The performance evaluation of the packet loss with the proposed SDP architecture shows significant changes from the setup without SDP architecture. The packet loss is significantly reduced when the SDP architecture is enabled, especially during attack scenarios. Specifically, SDP effectively reduces packet loss across all protocols during attacks. Without SDP, the average packet loss is about 10%, but with SDP, it's reduced to around 2%. Notably, there's no FTP packet loss with SDP during attacks. Figure 8 illustrates the packet loss percentage when the SDP is enabled and disabled under attack and normal conditions for respective RAS protocols. Another notable recorded observation is the reduction in RDP packet loss by 79.0% during attacks (0.0989–0.0208) and SSH packet loss by 80.0% (0.1002–0.0200). For precise clarification via statistical significance confirmation, the descriptive statistical information of the gateway packet loss results is given in the Appendix section by Fig. 24a (Normal with SDP disabled mode), b (Normal with SDP enabled mode), c (under attack with SDP disabled mode), and d (under attack with SDP enabled mode).

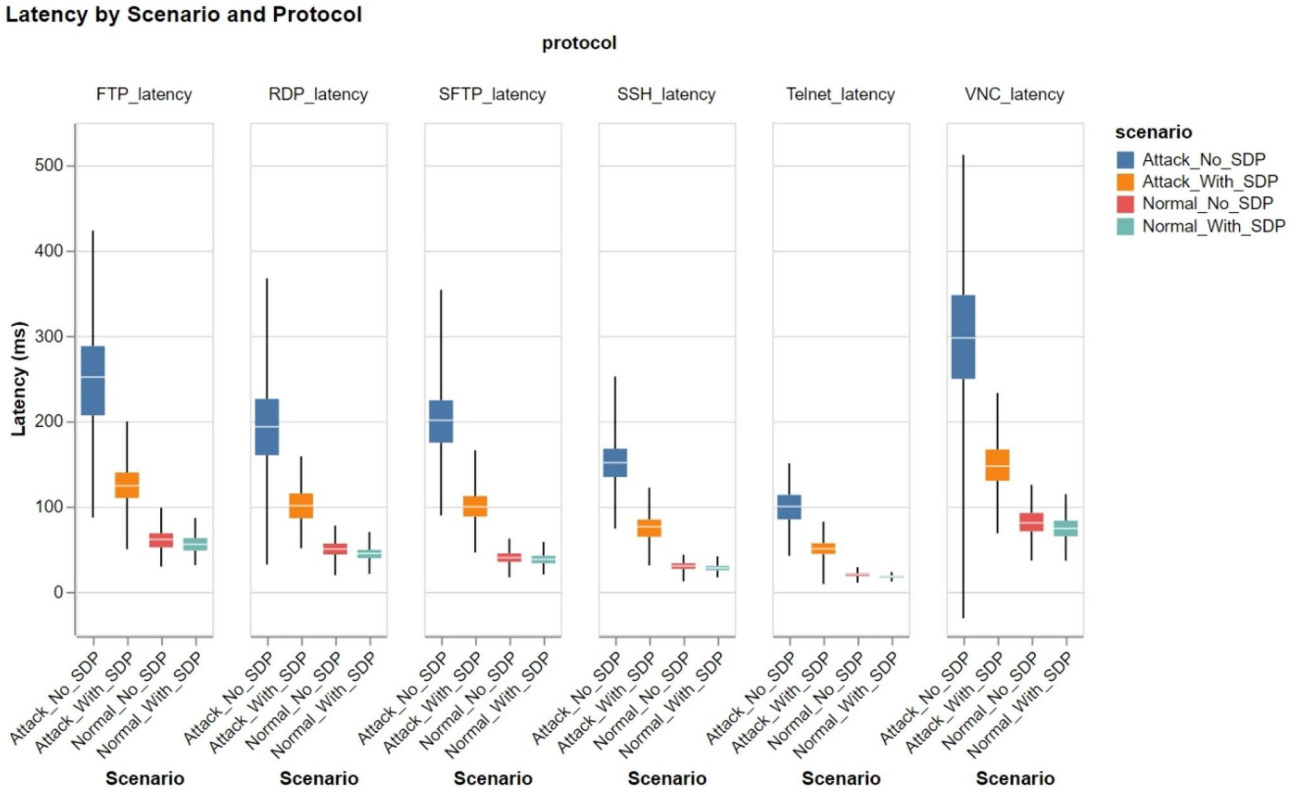


Fig. 7. Recorded latency for all RAS protocols when the SDP is enabled and disabled under attack and normal conditions.

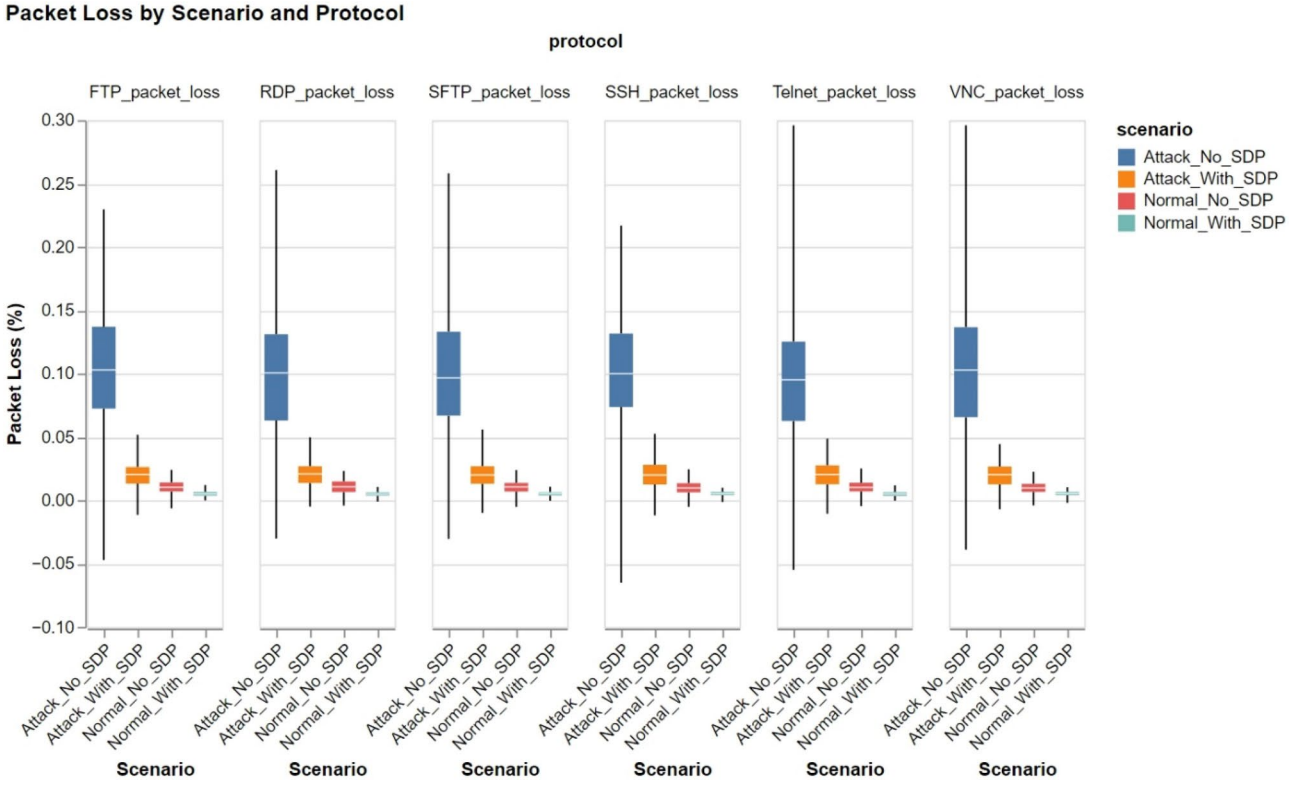


Fig. 8. Recorded packet loss for all RAS protocols when the SDP is enabled and disabled under attack and normal conditions.

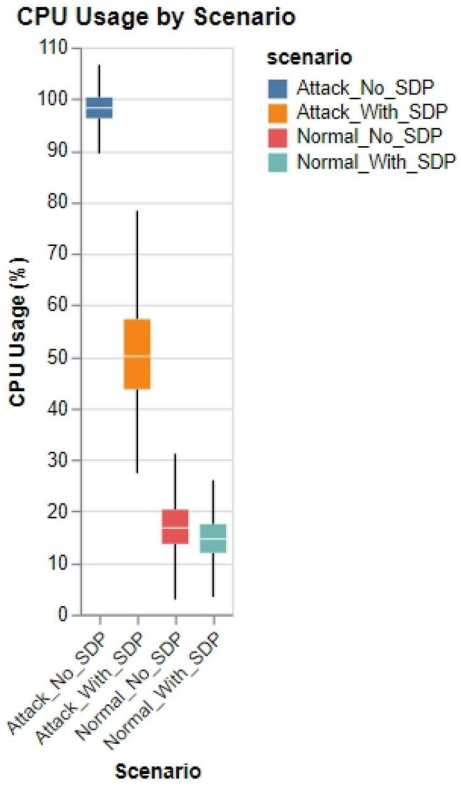


Fig. 9. CPU percentage usage of the SDP gateway when SDP is enabled and disabled under attack and normal conditions.

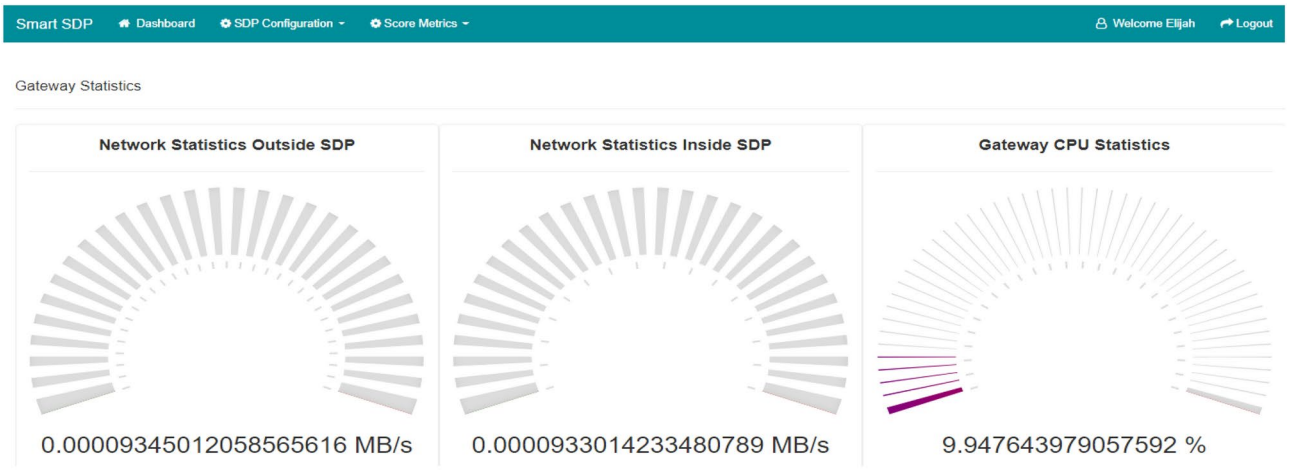
5. CPU Usage

The performance evaluation of the CPU usage with the proposed SDP architecture shows insignificant changes from the setup without SDP architecture. Generally, the CPU usage varies insignificantly and thus proves the viability of the proposed SDP architecture in an actual deployment environment. Figure 9 shows the CPU usage percentage when the SDP is enabled and disabled. On the one hand, with the SDP-disabled mode, the CPU usage has a median of 16.78 (Mean = 17.01, Standard deviation (STD) = 4.60, and Variance = 21.13). On the other hand, with the SDP-enabled mode, the CPU usage had a median of 14.67 (Mean = 14.71, Standard deviation (STD) = 3.93, and Variance = 15.43) under normal conditions. In this regard, in the context of resource consumption, SDP doesn't significantly change the average amount of data received and transmitted. Meanwhile, SDP reduces CPU usage during attacks, lowering it from an average of 98.25% to 50.41%, a reduction of around 48.7%, henceforth enabling sustained service availability. For precise clarification via statistical significance confirmation, the descriptive statistical information of the gateway CPU percentage usage results is given in the Appendix section by Fig. 24a (Normal with SDP disabled mode), b (Normal with SDP enabled mode), c (under attack with SDP disabled mode), and d (under attack with SDP enabled mode).

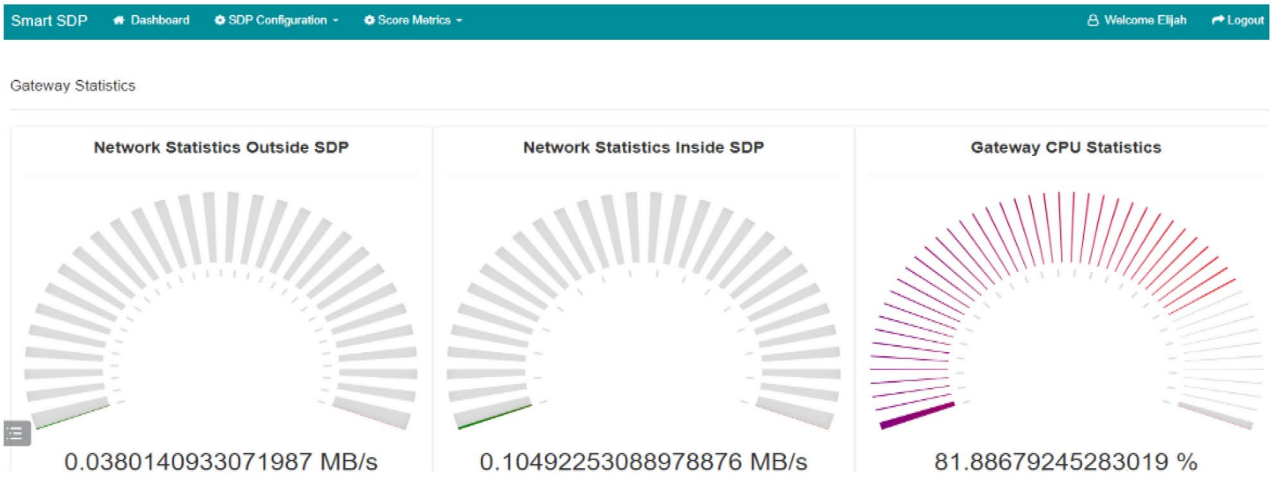
Without SDP architecture, the CPU usage of the gateway increased during BFA. At the same time, it was stable, and insignificant changes were recorded in the setup with the SDP-enabled architecture. Figures 10a and 11b show the CPU usage of the gateway during BFA in the SDP-enabled and disabled architecture, respectively. In the SDP-disabled architecture, high gateway CPU usage is due to a lack of ability to differentiate between legitimate and malicious requests. In this case, normal and malicious traffic are indistinguishable, leading to continuous high gateway CPU usage.

Overall, statistical analysis shows that SDP significantly mitigates attack impacts in the SDP architecture, reducing CPU usage, latency, and packet loss while improving or maintaining throughput. Under normal conditions, SDP primarily reduces CPU usage, latency, and packet loss, with a less pronounced effect on throughput. The confidence intervals in the data support these findings, highlighting the statistical significance of the differences observed under the four scenarios subjected to the SDP architecture for securing RAS.

Regarding DDoS on SDP architecture, Our proposed SDP model for securing RAS against DDoS attacks uses SPA combined with a deny-all firewall at the gateway. This approach blocks malicious traffic, making DoS/



(a)



(b)

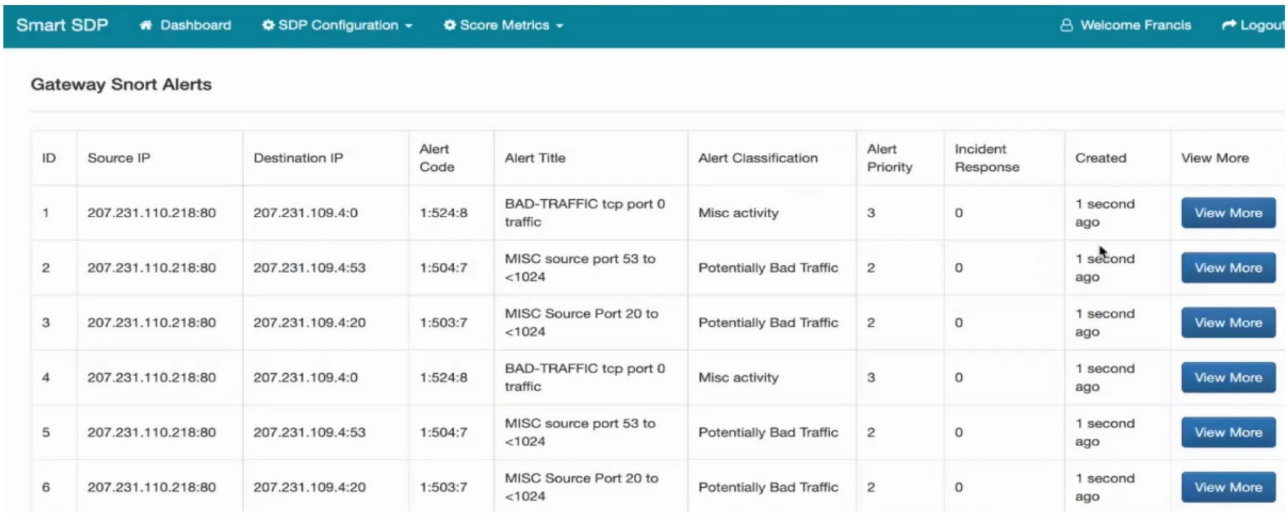
Fig. 10. (**a**) The Gateway's CPU usage during BFA with SDP architecture enabled (**b**) The Gateway's CPU usage during BFA with SDP architecture disabled.

DDoS attacks largely ineffective. SPA filters out unauthorized packets, allowing only legitimate traffic through, and forwards suspicious activity to Snort IDS for validation. This proactive rejection of DDoS traffic by the SDP Gateways places a minimal computational load on the network infrastructure.

In this regard, the proposed SDP model effectively protects RAS from DDoS attacks as the RAS access undergoes thorough authentication before granting access. The model counters well-known DDoS attacks by hiding services behind an SDP Gateway and employing a dynamic firewall mechanism to drop malicious packets quickly. Furthermore, the sessions are tracked by the *ConnTack* and Snort IDS in real-time, as shown in Fig. 11, analyzing and alerting insights of traffic behavior and enabling informed decision-making by the firewall.

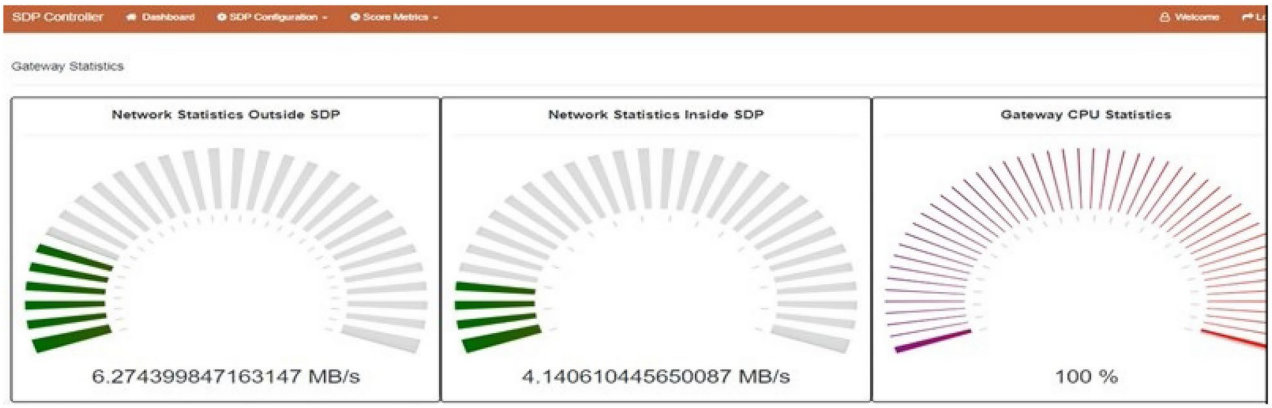
Figure 12a and b illustrate the effectiveness of the proposed SDP architecture. Figure 12a shows a network overwhelmed by DDoS attacks with SDP disabled mode, leading to resource unavailability for legitimate users. In contrast, 12b demonstrates how, with SDP enabled, the network remains stable as the SDP Gateways proactively reject DDoS traffic with minimal strain on the infrastructure. These results highlight the proposed SDP architecture's capability to mitigate DDoS attacks and maintain service availability.

6. Comparative Performance and Security with Existing BFA Mitigation Approaches

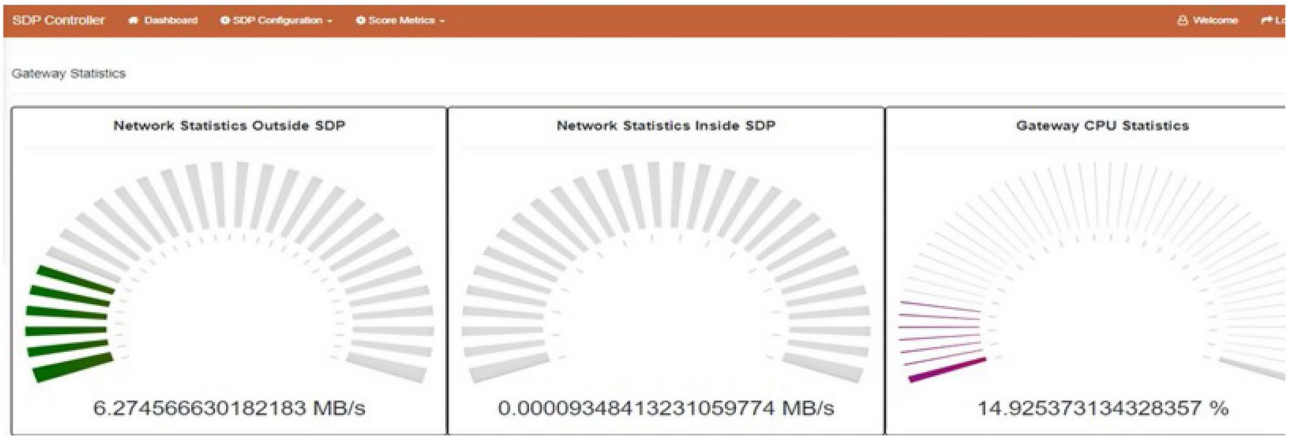


ID	Source IP	Destination IP	Alert Code	Alert Title	Alert Classification	Alert Priority	Incident Response	Created	View More
1	207.231.110.218:80	207.231.109.4:0	1:524:8	BAD-TRAFFIC tcp port 0 traffic	Misc activity	3	0	1 second ago	View More
2	207.231.110.218:80	207.231.109.4:53	1:504:7	MISC source port 53 to <1024	Potentially Bad Traffic	2	0	1 second ago	View More
3	207.231.110.218:80	207.231.109.4:20	1:503:7	MISC Source Port 20 to <1024	Potentially Bad Traffic	2	0	1 second ago	View More
4	207.231.110.218:80	207.231.109.4:0	1:524:8	BAD-TRAFFIC tcp port 0 traffic	Misc activity	3	0	1 second ago	View More
5	207.231.110.218:80	207.231.109.4:53	1:504:7	MISC source port 53 to <1024	Potentially Bad Traffic	2	0	1 second ago	View More
6	207.231.110.218:80	207.231.109.4:20	1:503:7	MISC Source Port 20 to <1024	Potentially Bad Traffic	2	0	1 second ago	View More

Fig. 11. SDP Gateway is enabled with Snort IDS to enhance real-time malicious network activity detection on RAS.



(a)



(b)

Fig. 12. (**a**) The SDP Gateway's CPU usage during DDoS attacks with SDP architecture enabled (**b**) The SDP Gateway's CPU usage during DDoS attacks with SDP architecture disabled.

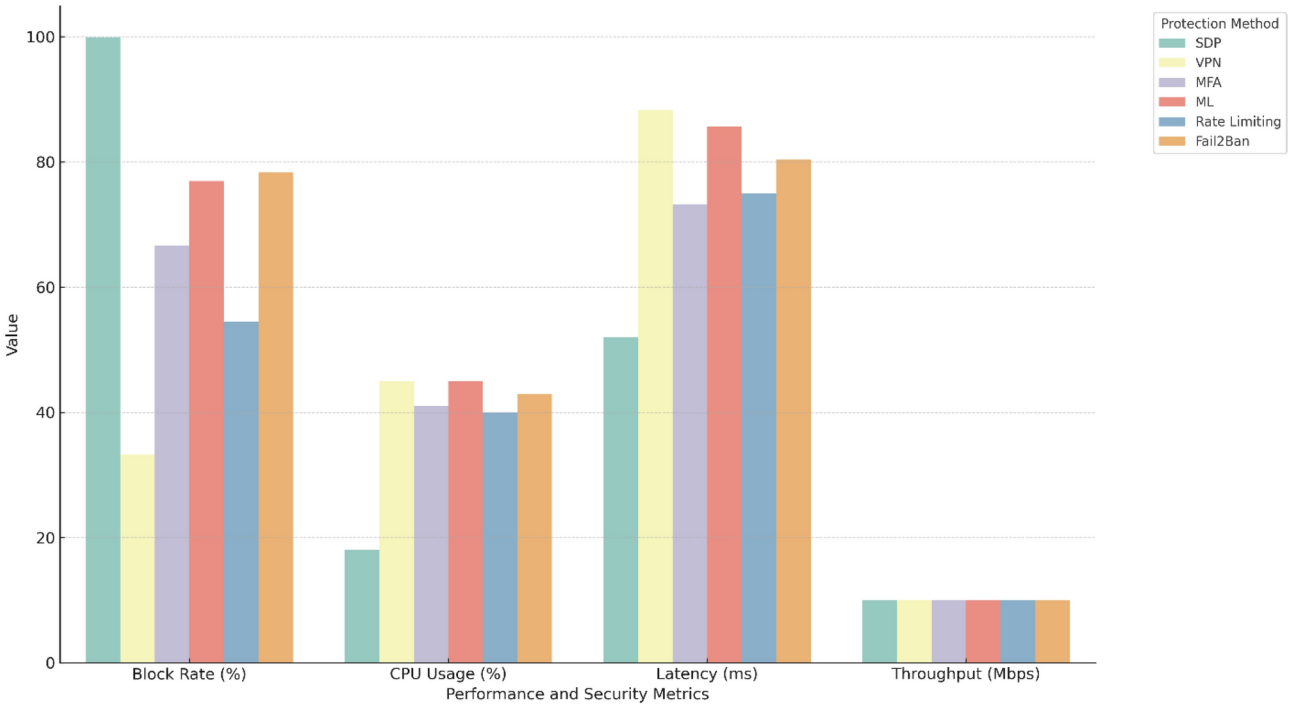


Fig. 13. Comparative performance and security analysis of the enhanced SDP versus existing BFA mitigation mechanisms.

To compare the proposed SDP architecture with the existing BFA mitigation approaches (covered in the related work Section), we conducted a comparative performance and security analysis of the enhanced SDP architecture versus conventional security methods for BFA mitigation: VPN, MFA, ML, Rate Limiting, and Fail2Ban.

Figure 13 demonstrates the SDP architecture's superiority over existing protection methods (VPN, MFA, ML, Rate Limiting, and Fail2Ban).

Specifically, the enhanced SDP achieved an exceptional average block rate of 99.9% against brute-force attacks, substantially surpassing VPN (33.23%), MFA (66.57%), ML (76.97%), rate Limiting (54.5%), and Fail2Ban (78.3%). Furthermore, SDP significantly reduced CPU utilization to 18%, compared to higher usage by traditional methods, such as VPN (45%), MFA (41%), ML (45%), rate Limiting (40%), and Fail2Ban (43%), indicating enhanced operational efficiency. SDP also showed the lowest latency at 52 ms, significantly better than VPN (88.3 ms), MFA (73.2 ms), ML (85.6 ms), rate limiting (75.0 ms), and Fail2Ban (80.4 ms), highlighting its superior responsiveness. Throughput remained consistently high (10 Mbps) across all evaluated methods, confirming that SDP effectively maintains robust security without compromising network performance. These results confirm SDP's effectiveness as a balanced, high-performance security solution ideal for enterprise deployment.

Discussion and limitation

This study presents an enhanced SDP framework tailored and designed to secure better RAS akin to BFA. It explicitly focuses on mitigating BFA, targeting widely used protocols like RDP, SSH, VNC, Telnet, FTP, and SFTP. The approach combines several security layers to limit unauthorized access attempts significantly. The combined approach includes service obfuscation via cloaking/hiding capability, SPA, and adaptive threat response mechanisms via snort IDPS. Tests conducted during the evaluation phase revealed notable gains. These gains include significantly reducing 48.7% CPU usage during ongoing attacks and enabling cost-effective scaling for enterprises while maintaining a seamless experience for legitimate users during attacks. One trade-off observed was a minor authentication delay, primarily attributed to the overhead from cryptographic verification and establishing secure communication tunnels. While this delay is generally tolerable in most corporate settings, optimizing the system further could improve its performance in latency-critical applications.

Despite its apparent benefits, the SDP solution has certain limitations. Firstly, the centralized nature of the controller could become a bottleneck in large-scale implementations. This insight suggests the need for further research into distributed or clustered controller designs. Additionally, integration complexities arise when incorporating this solution with legacy systems such as Active Directory, potentially hindering smooth organizational adoption. The current security framework presumes SPA token confidentiality and does not explicitly cover scenarios involving compromised internal credentials or token exposure, indicating the necessity for supplemental security measures. Thus far, the formal security analyses have simplified several assumptions, notably omitting a detailed examination of side-channel vulnerabilities. Consequently, future investigations should involve more rigorous verification using sophisticated analytical tools (such as ProVerif or Tamarin).

Furthermore, given the ever-evolving nature of cyber threats—such as AI-assisted credential cracking and coordinated, slow-paced brute-force attacks by distributed botnets—it becomes imperative to incorporate advanced behavioral analytics, anomaly detection systems, and reputation-based IP filtering technologies. Additionally, the extensive logging required for threat detection poses compliance challenges concerning privacy regulations like GDPR. Future implementations must thus carefully balance detailed forensic logging with privacy safeguards such as anonymization and data retention policies to maintain user trust and regulatory adherence.

The findings from this study provide practical advice for implementing zero-trust security principles effectively within operational networks. The SDP model aligns with zero-trust frameworks by proactively hiding services using SPA, effectively preventing BFA with minimal latency and reducing computational overhead. In contrast to conventional reactive security measures (like fail2ban and rate limiting) that address attacks only after initiation, SDP preemptively blocks threats, significantly lowering false positive rates and user inconvenience. Moreover, the SDP design surpasses the capabilities of traditional static defenses such as firewalls and VPNs, which are susceptible to encrypted attacks and incur notable latency and processing overhead.

A recommendation building upon these understanding and insights is to achieve end-to-end zero-trust security by integrating SDP architecture with other supporting technologies, such as MFA for robust identity authentication and ML-based anomaly detection systems for real-time response against emerging attacks. The combined solution provides security layers, thus overcoming the scalability and operational performance limitations normally inherent in traditional protective strategies. These real-world insights aim to enlighten practitioners on deploying robust, scalable, and privacy-friendly zero-trust security architectures for diversified and dynamic enterprise networks.

Conclusion and future work

Preventing BFA on RAS (RDP, SSH, VNC, Telnet, FTP, and SFTP) continues to be challenging in light of the increased adoption of remote work and the evolving cybersecurity threats. This paper introduces an enhanced SDP architecture intending to actively secure RAS by leveraging SPA, dynamic access control, and active threat mitigation capabilities. We validated the proposed practical, lightweight prototype (implemented in Docker) to effectively thwart BFA and DDoS attacks (as realistic testing scenarios targeting RAS) using rigorous formal analysis and thorough empirical evaluations. Cryptographically obscuring services from unauthorized visibility and dynamically blocking malicious access attempts, the architecture maintains strong security without significantly impacting legitimate user experience or operational performance. However, further research is needed to address outstanding challenges related to scalability, seamless integration with legacy infrastructure, and continuously adapting to new cyber threats. Future work should focus on expanding SDP compatibility to additional protocols and post-quantum schemes/algorithms across a myriad of verticals, such as those used in IoT and web-based systems; employing adaptive security measures, such as AI-assisted techniques to catch up with emerging threats and improve user experience without compromising security and privacy; conducting comprehensive formal verification of cryptographic protocols using advanced tools like Tamarin and ProVerif; enhancing user experience by introducing streamlined recovery mechanisms for legitimate users inadvertently affected by security measures; and rigorously evaluating system scalability and robustness through simulation across a myriad of advanced attacks. Ultimately, the enhanced SDP architecture introduced here provides a solid foundation for securing RAS, with ongoing improvements in intelligent authentication strategies expected to further strengthen cybersecurity practices in diverse and evolving environments.

Data availability

All data will be made available upon reasonable request to the corresponding author.

Appendix

See Appendix Figs. 14, 15, 16, 17, 18, 19, 20, 21, 22, 23 and 24.

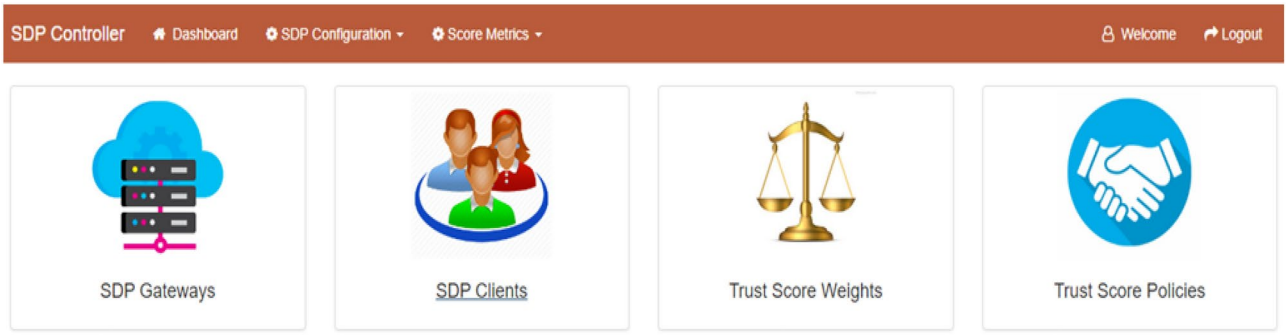


Fig. 14. SDP controller interface main components.

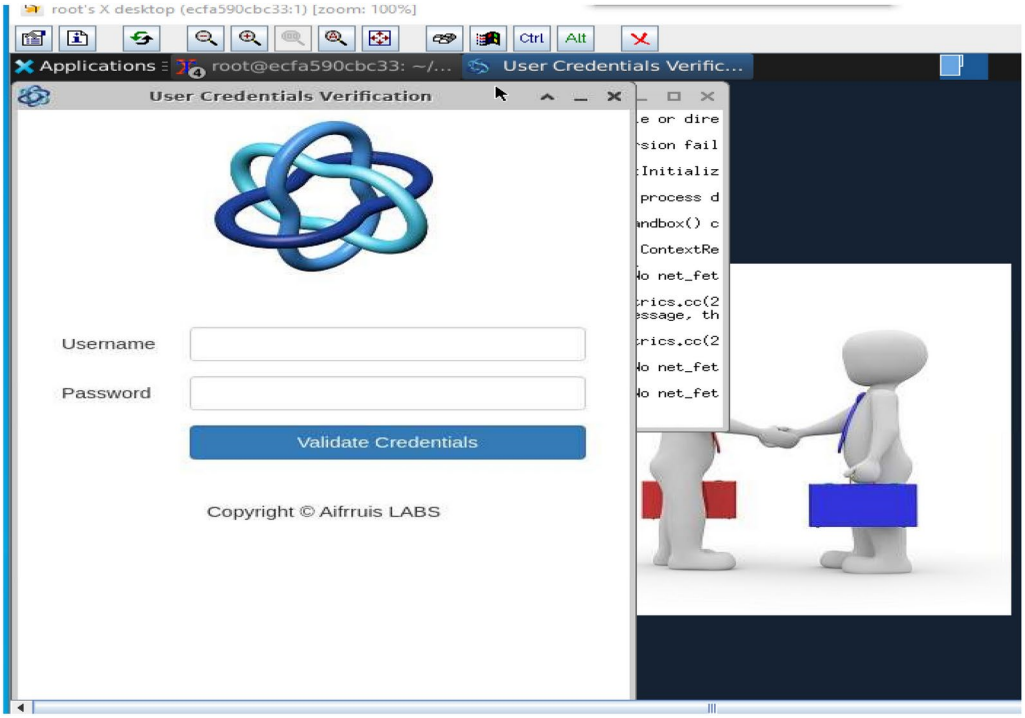


Fig. 15. Desktop SDP client application.

```
root@805b42a70a3e:/# nmap -v -A 172.17.0.3
Starting Nmap 7.80 ( https://nmap.org ) at 2022-02-16 19:06 EAT
NSE: Loaded 151 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Initiating ARP Ping Scan at 19:06
Scanning 172.17.0.3 [1 port]
Completed ARP Ping Scan at 19:06, 0.01s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 19:06
Completed Parallel DNS resolution of 1 host. at 19:06, 0.00s elapsed
Initiating SYN Stealth Scan at 19:06
Scanning 172.17.0.3 [1000 ports]
Completed SYN Stealth Scan at 19:06, 1.23s elapsed (1000 total ports)
Initiating Service scan at 19:06
Initiating OS detection (try #1) against 172.17.0.3
Retrying OS detection (try #2) against 172.17.0.3
NSE: Script scanning 172.17.0.3.
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Initiating NSE at 19:06
Completed NSE at 19:06, 0.00s elapsed
Nmap scan report for 172.17.0.3
Host is up (0.00012s latency).
Not shown: 993 closed ports
PORT      STATE     SERVICE        VERSION
21/tcp    filtered ftp
22/tcp    filtered ssh
23/tcp    filtered telnet
80/tcp    filtered http
3389/tcp filtered ms-wbt-server
5901/tcp filtered vnc-1
6001/tcp filtered X11:1
```

(a)

```
root@805b42a70a3e:/# nmap -v -A 172.17.0.3
Starting Nmap 7.80 ( https://nmap.org ) at 2022-02-16 19:00 EAT
NSE: Loaded 151 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 19:00
Completed NSE at 19:00, 0.00s elapsed
Initiating NSE at 19:00
Completed NSE at 19:00, 0.00s elapsed
Initiating NSE at 19:00
Completed NSE at 19:00, 0.00s elapsed
Initiating ARP Ping Scan at 19:00
Scanning 172.17.0.3 [1 port]
Completed ARP Ping Scan at 19:00, 0.01s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 19:00
Completed Parallel DNS resolution of 1 host. at 19:00, 0.00s elapsed
Initiating SYN Stealth Scan at 19:00
Scanning 172.17.0.3 [1000 ports]
Discovered open port 23/tcp on 172.17.0.3
Discovered open port 3389/tcp on 172.17.0.3
Discovered open port 21/tcp on 172.17.0.3
Discovered open port 80/tcp on 172.17.0.3
Discovered open port 22/tcp on 172.17.0.3
Discovered open port 5901/tcp on 172.17.0.3
Discovered open port 6001/tcp on 172.17.0.3
Completed SYN Stealth Scan at 19:00, 0.04s elapsed (1000 total ports)
Initiating Service scan at 19:00
Scanning 7 services on 172.17.0.3
Completed Service scan at 19:00, 11.03s elapsed (7 services on 1 host)
Initiating OS detection (try #1) against 172.17.0.3
NSE: Script scanning 172.17.0.3.
Initiating NSE at 19:00
```

(b)

Fig. 16. (**a**) Nmap port scanning and identifying RAS with SDP-enabled architecture (**b**) Nmap port scanning and identifying RAS with SDP-disabled architecture.

```
root@805b42a70a3e:/home# hydra -l root -P /home/word_list/10-million-password-list-top-1000000.txt -t 1 172.17.0.3 rdp -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-02-15 13:40:17
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 1 task per 1 server, overall 1 task, 999998 login tries (l:1/p:999998), ~999998 tries per task
[DATA] attacking rdp://172.17.0.3:3389/
[ATTEMPT] target 172.17.0.3 - login "root" - pass "123456" - 1 of 999998 [child 0] (0/0)
[ERROR] freerdp: The connection failed to establish.
[ATTEMPT] target 172.17.0.3 - login "root" - pass "password" - 2 of 999999 [child 0] (0/1)
[ERROR] freerdp: The connection failed to establish.
[ATTEMPT] target 172.17.0.3 - login "root" - pass "12345678" - 3 of 1000000 [child 0] (0/2)
[ERROR] freerdp: The connection failed to establish.
[ATTEMPT] target 172.17.0.3 - login "root" - pass "qwerty" - 4 of 1000001 [child 0] (0/3)
```

(a)

```
root@805b42a70a3e:/home# hydra -l root -P /home/word_list/10-million-password-list-top-1000000.txt -t 1 172.17.0.3 rdp -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-02-15 13:29:16
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 999998 login tries (l:1/p:999998), ~999998 tries per task
[DATA] attacking rdp://172.17.0.3:3389/
[ATTEMPT] target 172.17.0.3 - login "root" - pass "123456" - 1 of 999998 [child 0] (0/0)
[3389][rdp] host: 172.17.0.3   login: root   password: 123456
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2022-02-15 13:29:28
root@805b42a70a3e:/home#
```

(b)

Fig. 17. (**a**) BFA on RDP service with SDP architecture enabled (**b**) BFA on RDP service with SDP architecture disabled.

```
root@805b42a70a3e:/home# hydra -s 5901 -P /home/word_list/10-million-password-list-top-1000000.txt -t 1 172.17.0.3 vnc -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-02-14 16:38:55
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 999998 login tries (l:1/p:999998), ~999998 tries per task
[DATA] attacking vnc://172.17.0.3:5901/
[ATTEMPT] target 172.17.0.3 - login "" - pass "123456" - 1 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "password" - 2 of 999999 [child 0] (0/1)
[STATUS] 2.00 tries/min, 2 tries in 00:01h, 999997 to do in 8333:19h, 1 active
[ATTEMPT] target 172.17.0.3 - login "" - pass "12345678" - 3 of 1000000 [child 0] (0/2)
```

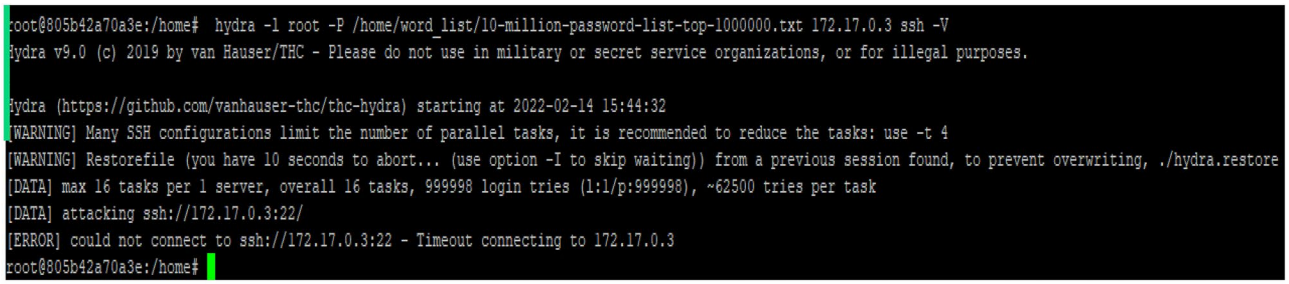
(a)

```
root@805b42a70a3e:/home# hydra -s 5901 -P /home/word_list/10-million-password-list-top-1000000.txt -t 1 172.17.0.3 vnc -V
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal purposes.

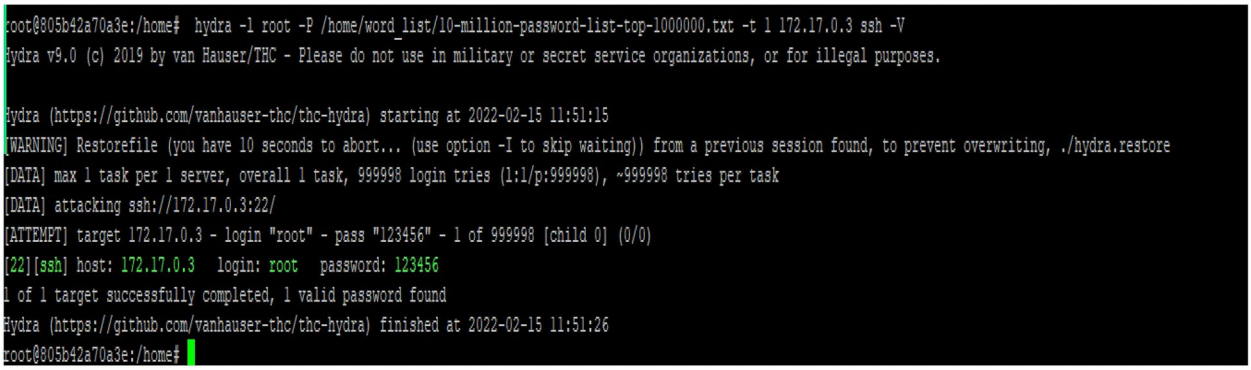
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2022-02-14 16:32:43
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 1 task per 1 server, overall 1 task, 999998 login tries (l:1/p:999998), ~999998 tries per task
[DATA] attacking vnc://172.17.0.3:5901/
[ATTEMPT] target 172.17.0.3 - login "" - pass "123456" - 1 of 999998 [child 0] (0/0)
[5901][vnc] host: 172.17.0.3   password: 123456
[ATTEMPT] target 172.17.0.3 - login "" - pass "password" - 2 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "12345678" - 3 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "qwerty" - 4 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "123456789" - 5 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "12345" - 6 of 999998 [child 0] (0/0)
[ATTEMPT] target 172.17.0.3 - login "" - pass "1234" - 7 of 999998 [child 0] (0/0)
```

(b)

Fig. 18. (**a**) BFA on VNC service with SDP architecture enabled (**b**) BFA on VNC service with SDP architecture disabled.

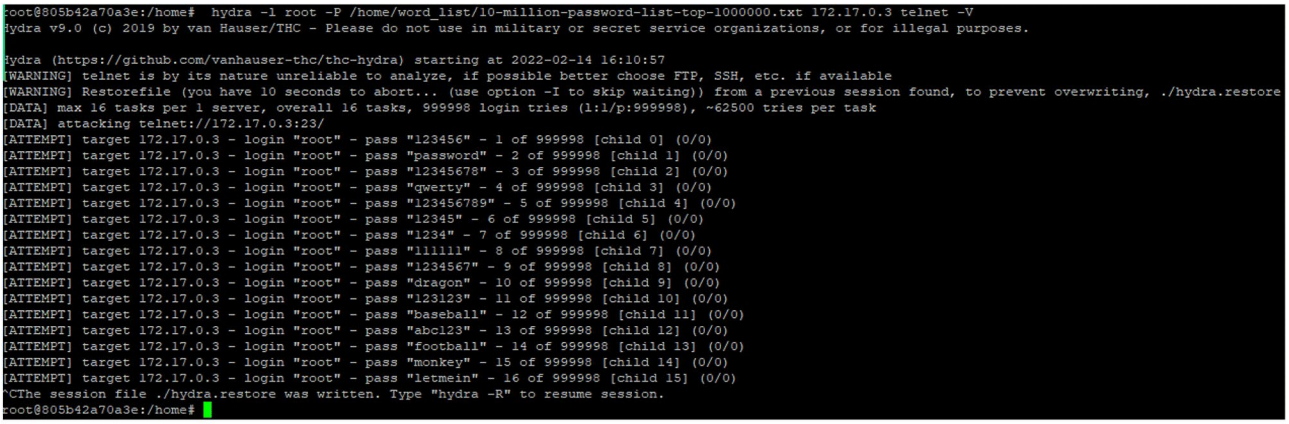


(a)

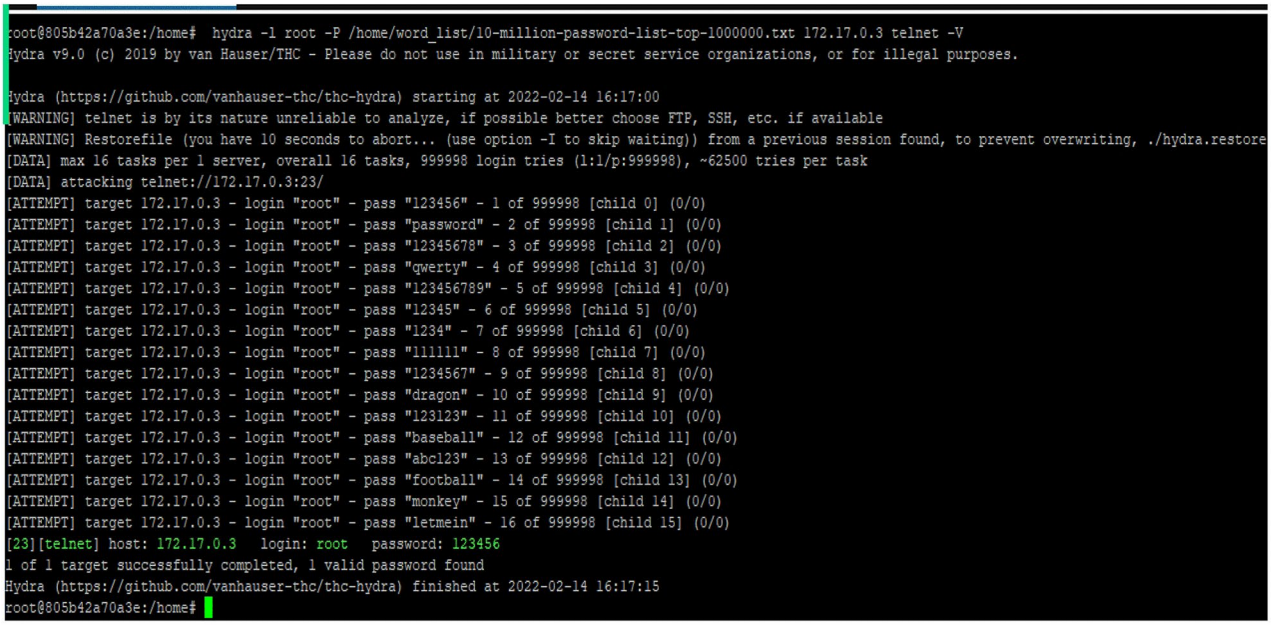


(b)

Fig. 19. (**a**) BFA on SSH service with SDP architecture enabled (**b**) BFA on SSH service with SDP architecture disabled.

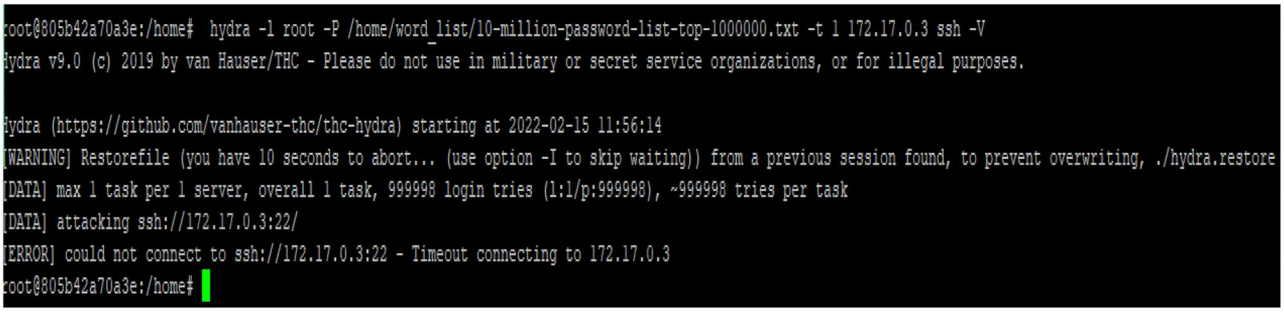


(a)

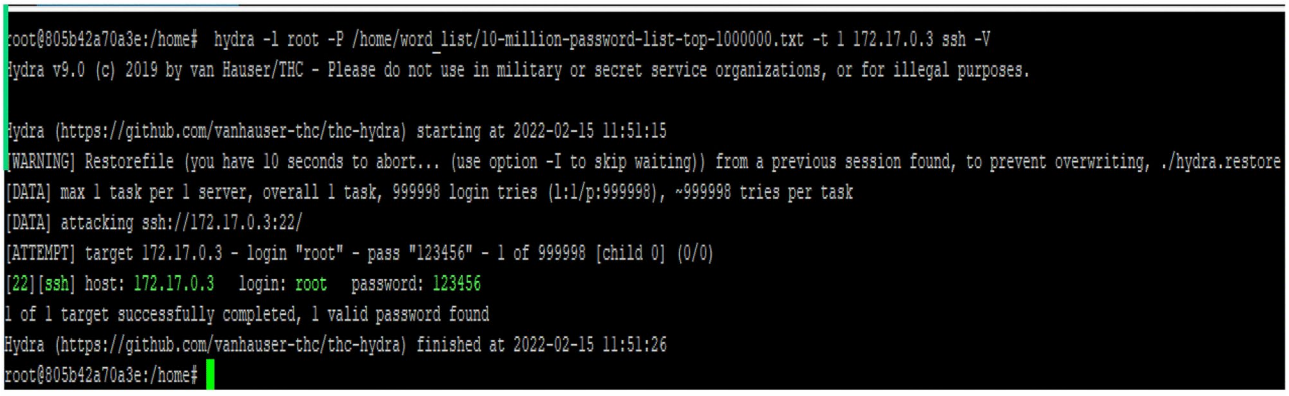


(b)

Fig. 20. (**a**) BFA on Telnet service with SDP architecture enabled (**b**) BFA on Telnet service with SDP architecture disabled.

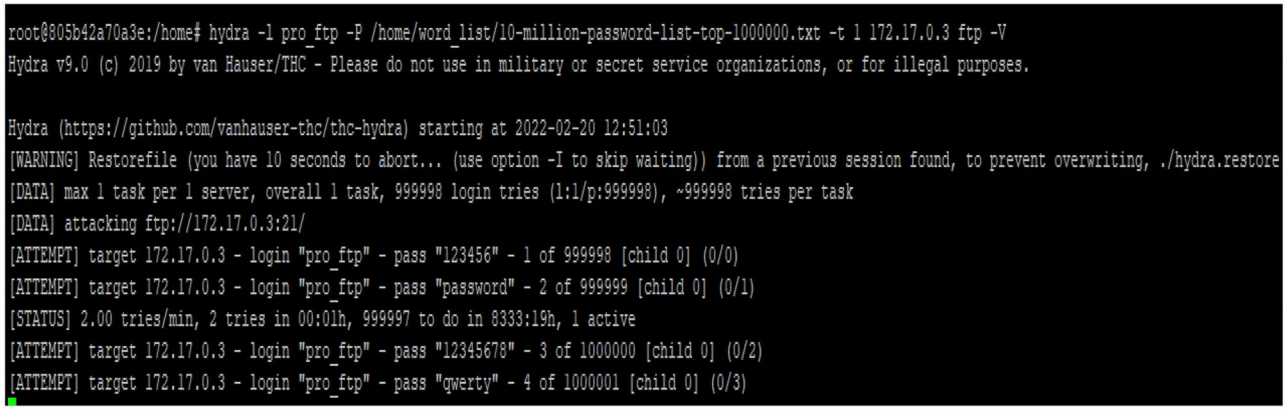


(a)

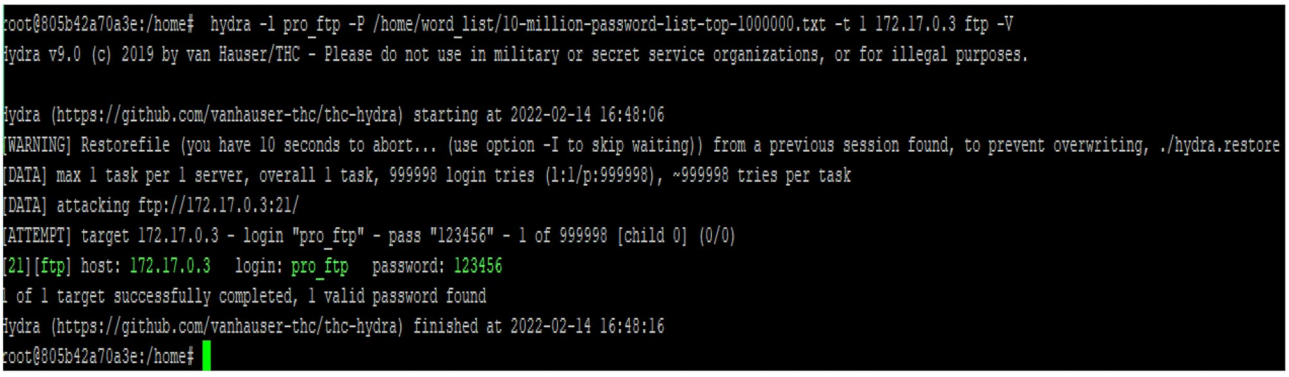


(b)

Fig. 21. (**a**) BFA on SFTP service with SDP architecture enabled (**b**) BFA on SFTP service with SDP architecture disabled.

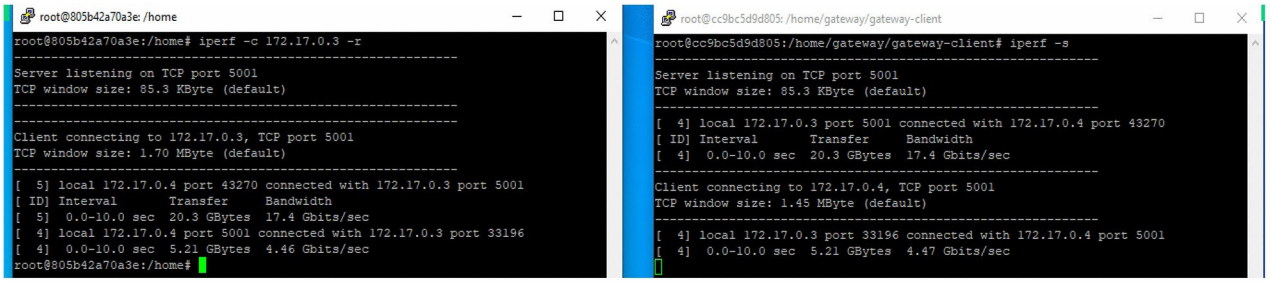


(a)

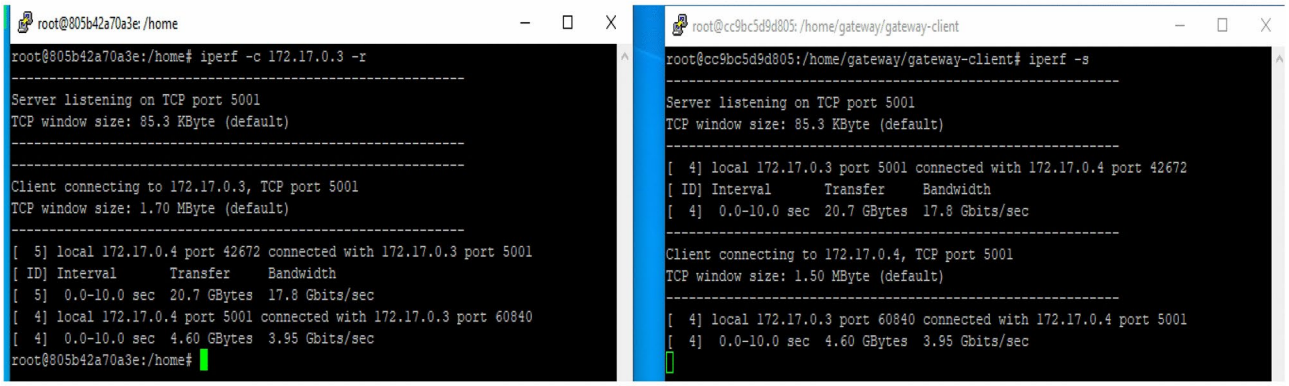


(b)

Fig. 22. (**a**) BFA on FTP service with SDP architecture enabled (**b**) BFA on FTP service with SDP architecture disabled.



(a)



(b)

Fig. 23. (**a**) Throughput measured by iperf with SDP architecture enabled (**b**) Throughput measured by iperf with SDP architecture disabled.

Metric	Mean	Std Dev	Median	Q1	Q3	Min	Max	95% CI Lower	95% CI Upper
rxCount	0.0017	0.0005	0.0017	0.0013	0.002	-0.0001	0.003	0.0016	0.0017
txCount	0.0009	0.0003	0.0009	0.0007	0.0011	0.0001	0.0021	0.0009	0.001
cpuPercent	17.0095	4.5965	16.7781	13.7065	20.3263	2.9146	31.1688	16.533	17.4859
RDP_latency	50.1366	10.0013	50.086	43.9167	56.6077	19.4424	77.6085	49.1	51.1732
RDP_throughput	10.1532	1.9435	10.1872	8.6919	11.4784	5.207	15.9324	9.9517	10.3546
SSH_latency	30.2218	5.2806	30.374	26.9962	33.5351	12.3524	43.5702	29.6745	30.7691
SSH_throughput	5.0209	1.023	4.9674	4.3226	5.7083	1.8923	7.8418	4.9148	5.1269
VNC_latency	81.4871	15.2915	80.8172	71.0395	92.4062	36.6948	125.4895	79.9021	83.072
VNC_throughput	19.9235	3.8346	20.014	17.4338	22.3757	5.7515	31.3971	19.526	20.3209
Telnet_latency	19.9882	2.9187	20.0557	18.2454	21.7228	10.8342	28.8513	19.6857	20.2907
Telnet_throughput	1.0196	0.1787	1.026	0.8993	1.1501	0.5019	1.5189	1.0011	1.0381
FTP_latency	60.848	12.0306	61.4158	52.0953	68.6643	29.5411	98.6036	59.6011	62.095
FTP_throughput	15.0181	3.0101	14.9568	12.8457	16.9186	7.9076	23.8835	14.7061	15.3301
SFTP_latency	40.2559	7.3728	40.0595	35.1954	45.038	16.9407	62.4419	39.4917	41.0201
SFTP_throughput	8.1004	1.7174	8.0961	7.013	9.1948	1.8465	13.7641	7.9223	8.2784
RDP_packet_loss	0.0104	0.0055	0.0106	0.0064	0.0148	-0.0043	0.0231	0.0099	0.011
SSH_packet_loss	0.0095	0.0052	0.0097	0.0062	0.0134	-0.0053	0.0244	0.009	0.01
VNC_packet_loss	0.0097	0.0048	0.0097	0.0065	0.0129	-0.0042	0.0224	0.0092	0.0102
Telnet_packet_loss	0.0101	0.0052	0.0101	0.007	0.0138	-0.0047	0.025	0.0096	0.0106
FTP_packet_loss	0.0102	0.005	0.0102	0.0069	0.014	-0.0065	0.024	0.0097	0.0108
SFTP_packet_loss	0.0101	0.0048	0.0105	0.0068	0.0136	-0.0053	0.0238	0.0096	0.0106

(a)

Metric	Mean	Std Dev	Median	Q1	Q3	Min	Max	95% CI Lower	95% CI Upper
rxCount	0.0017	0.0005	0.0017	0.0014	0.002	0.0004	0.003	0.0017	0.0018
txCount	0.001	0.0003	0.001	0.0007	0.0012	-0.0001	0.0018	0.0009	0.001
cpuPercent	14.7116	3.9276	14.6661	11.9652	17.5216	3.4051	26.069	14.3045	15.1187
RDP_latency	44.6851	7.7683	45.0163	39.4701	49.2406	21.0201	70.1812	43.8799	45.4902
RDP_throughput	10.0315	2.1972	9.8951	8.5324	11.5397	3.3324	15.6508	9.8037	10.2592
SSH_latency	27.9165	3.9038	28.0579	25.254	30.4192	16.8501	41.6444	27.5119	28.3211
SSH_throughput	4.9988	0.9974	5.0208	4.308	5.6352	2.3337	7.9137	4.8954	5.1022
VNC_latency	74.3799	12.6342	74.2578	65.1114	83.505	36.5134	114.5631	73.0704	75.6894
VNC_throughput	19.7127	4.2103	19.8078	16.8798	22.4826	6.225	33.6957	19.2763	20.1491
Telnet_latency	17.9302	1.9775	18.1229	16.5741	19.152	12.015	23.392	17.7252	18.1351
Telnet_throughput	0.9989	0.1989	1.0011	0.8568	1.1254	0.1659	1.5382	0.9783	1.0195
FTP_latency	55.6409	10.2594	55.7673	48.423	62.9546	31.2305	86.6352	54.5775	56.7043
FTP_throughput	15.0001	2.9443	14.9466	12.9916	16.7486	7.9596	24.1041	14.695	15.3053
SFTP_latency	38.2762	6.6885	38.0919	33.4688	42.3396	20.3836	58.5869	37.583	38.9695
SFTP_throughput	8.0534	1.6078	8.1202	6.9106	9.1806	2.9546	14.2042	7.8867	8.22
RDP_packet_loss	0.0049	0.002	0.0048	0.0035	0.0062	-0.0011	0.0105	0.0047	0.0051
SSH_packet_loss	0.0051	0.0021	0.0051	0.0039	0.0065	-0.0014	0.0098	0.0049	0.0053
VNC_packet_loss	0.0052	0.0019	0.0052	0.0039	0.0065	-0.0021	0.0102	0.005	0.0054
Telnet_packet_loss	0.0049	0.0021	0.005	0.0033	0.0063	-0.0004	0.0117	0.0046	0.0051
FTP_packet_loss	0.0051	0.0021	0.005	0.0035	0.0065	-0.0001	0.0119	0.0049	0.0053
SFTP_packet_loss	0.005	0.002	0.0051	0.0037	0.0063	-0.0005	0.0106	0.0048	0.0052

(b)

Fig. 24. (**a**–**d**) The descriptive statistical information of the RAS protocols and Resource Consumption of the gateway CPU.

Metric	Mean	Std Dev	Median	Q1	Q3	Min	Max	95% CI Lower	95% CI Upper
rxCount	2.2001	0.2969	2.2126	1.9955	2.3843	1.3632	3.2589	2.1693	2.2308
txCount	1.2927	0.2933	1.2802	1.094	1.4894	0.6007	2.0873	1.2623	1.3231
cpuPercent	98.2465	3.1201	98.2727	96.2511	100.2923	89.4847	106.6435	97.9231	98.5699
RDP_latency	194.3886	49.4077	193.5109	160.1496	225.9952	32.0063	367.5015	189.2675	199.5096
RDP_throughput	1.9971	0.4841	2.0272	1.6858	2.3322	0.4431	3.3934	1.947	2.0473
SSH_latency	151.6494	28.5264	151.3007	134.564	167.7716	74.1928	252.3992	148.6927	154.6062
SSH_throughput	1.0016	0.1832	1.0105	0.8825	1.1224	0.482	1.5018	0.9826	1.0205
VNC_latency	298.7569	77.3869	297.4752	249.6075	347.8213	-30.9587	512.0533	290.7358	306.7779
VNC_throughput	4.917	0.9913	4.9958	4.2668	5.5715	2.1132	8.9507	4.8143	5.0198
Telnet_latency	99.6745	20.819	100.0559	84.9047	113.6508	42.279	150.4885	97.5166	101.8324
Telnet_throughput	0.5015	0.1033	0.4957	0.4295	0.5756	0.1785	0.808	0.4908	0.5122
FTP_latency	248.1133	60.9754	251.6865	206.8538	287.8587	86.9989	423.4388	241.7933	254.4333
FTP_throughput	3.1021	0.7276	3.0709	2.6238	3.6456	1.0155	5.3746	3.0267	3.1776
SFTP_latency	199.8295	40.5823	201.1217	174.88	224.4842	89.4489	353.9849	195.6232	204.0358
SFTP_throughput	2.0009	0.3815	1.9982	1.7383	2.2421	0.9381	3.2418	1.9614	2.0405
RDP_packet_loss	0.0989	0.0518	0.1004	0.0629	0.131	-0.0303	0.2606	0.0936	0.1043
SSH_packet_loss	0.1002	0.0484	0.0999	0.0735	0.1316	-0.0651	0.2168	0.0952	0.1052
VNC_packet_loss	0.1031	0.0522	0.1027	0.0655	0.1366	-0.0391	0.2959	0.0977	0.1085
Telnet_packet_loss	0.0929	0.0495	0.095	0.0625	0.1252	-0.055	0.2959	0.0878	0.098
FTP_packet_loss	0.1033	0.048	0.1028	0.0722	0.1369	-0.0473	0.2294	0.0983	0.1083
SFTP_packet_loss	0.1001	0.049	0.0965	0.0668	0.1329	-0.0306	0.258	0.095	0.1052

(c)

Metric	Mean	Std Dev	Median	Q1	Q3	Min	Max	95% CI Lower	95% CI Upper
rxCount	0.5019	0.1032	0.5066	0.4374	0.5615	0.2149	0.8611	0.4912	0.5126
txCount	0.3081	0.1017	0.3027	0.2372	0.3731	0.0344	0.6308	0.2976	0.3187
cpuPercent	50.4085	9.5419	50.0852	43.7075	57.2695	27.3956	78.3276	49.4195	51.3975
RDP_latency	100.7643	20.5176	100.9515	86.2821	115.3821	51.0629	158.835	98.6377	102.8909
RDP_throughput	7.9097	1.5998	7.9276	6.8666	9.0152	3.4847	12.7478	7.7438	8.0755
SSH_latency	75.0557	14.7207	76.337	64.6065	84.6922	30.9102	122.1209	73.53	76.5815
SSH_throughput	4.0102	0.8112	4.0508	3.3832	4.5499	0.9739	6.7413	3.9262	4.0943
VNC_latency	149.017	28.9081	147.3305	130.2427	166.8199	68.6582	233.2517	146.0207	152.0133
VNC_throughput	15.1613	3.2052	15.1848	12.9921	17.3337	6.9328	27.6703	14.8291	15.4935
Telnet_latency	50.7687	9.6518	50.5235	44.4261	57.017	9.1521	82.3819	49.7683	51.769
Telnet_throughput	0.8047	0.1477	0.7989	0.704	0.9111	0.4238	1.1781	0.7894	0.82
FTP_latency	125.4485	23.1635	124.2942	109.9359	139.9017	49.8954	199.75	123.0476	127.8493
FTP_throughput	9.9024	1.9938	9.8444	8.5433	11.2059	4.2304	15.8917	9.6957	10.109
SFTP_latency	100.6788	20.4252	99.7213	88.2481	112.2822	46.1464	165.8701	98.5617	102.7958
SFTP_throughput	6.0163	1.1402	6.0583	5.3422	6.8651	2.585	9.1053	5.8981	6.1345
RDP_packet_loss	0.0208	0.0096	0.0207	0.0136	0.0268	-0.0051	0.0496	0.0198	0.0217
SSH_packet_loss	0.02	0.0105	0.0198	0.0124	0.028	-0.012	0.0524	0.0189	0.0211
VNC_packet_loss	0.0197	0.0105	0.0201	0.0125	0.0265	-0.0072	0.0442	0.0186	0.0208
Telnet_packet_loss	0.0195	0.0102	0.0201	0.0126	0.0275	-0.0107	0.0485	0.0184	0.0206
FTP_packet_loss	0.0199	0.01	0.0202	0.0131	0.0262	-0.0116	0.0516	0.0188	0.0209
SFTP_packet_loss	0.0196	0.0105	0.0199	0.013	0.0268	-0.01	0.0557	0.0185	0.0207

(d)

Fig. 24. (continued)

Received: 28 January 2025; Accepted: 2 May 2025
Published online: 28 May 2025

References

1. Magaña, E., Sesma, I., Morató, D. & Izal, M. Remote access protocols for Desktop-as-a-Service solutions. *PLoS ONE* **14**, e0207512. https://doi.org/10.1371/journal.pone.0207512 (2019).
2. Liao, J., Zhang, H., Xu, Y., Li, J. & Timorabadi, H. S. WIP—iCtrl—A remote login software application. In *ASEE Annu. Conf. Expo. Conf. Proc.* https://doi.org/10.18260/1-2--40631 (2022).
3. van der Kleut, J. Remote computer access: What is it and what are the risks? | NortonLifeLock. https://us.norton.com/internetsecurity-how-to-remote-computer-access.html#remote. Accessed 18 Feb 2022 (2020).
4. Saito, S., Maruhashi, K., Takenaka, M. & Torii, S. TOPASE: Detection and prevention of brute force attacks with disciplined IPs from IDS logs. *J. Inf. Process.* **24**(2), 217–226. https://doi.org/10.2197/ipsjjip.24.217 (2016).

5. Mohammed, T., Toderick, L., Ozan, E. & Lunsford, P. A case study on the SSH brute force dictionary attack on information technology labs. In *IMSCI 2007 - International Multi-Conference on Society, Cybernetics and Informatics, Proceedings* Vol. 2, 190–195 (2007).
6. Kakarla, T., Mairaj, A. & Javaid, A. Y. A real-world password cracking demonstration using open source tools for instructional use. In *IEEE Int. Conf. Electro Inf. Technol.* 387–391. https://doi.org/10.1109/EIT.2018.8500257 (2018).
7. Chethana Datta, J., Ananya, S., Deepak, M., Mungara, N. & Sarasvathi, V. Framework for brute-force attack detection using federated learning. In *Lect. Notes Inst. Comput. Sci. Soc. Telecommun. Eng. LNICST* Vol. 601, 64–73. https://doi.org/10.1007/978-3-031-81168-5_7 (2025).
8. Hamza, A. A. & Surayh Al-Janabi, R. J. Detecting brute force attacks using machine learning. In *BIO Web Conf.* Vol. 97, 00045. https://doi.org/10.1051/bioconf/20249700045 (2024).
9. Arghire, I. Cisco: Multiple VPN, SSH Services Targeted in Mass Brute-Force Attacks—SecurityWeek. https://www.securityweek.com/cisco-multiple-vpn-ssh-services-targeted-in-mass-brute-force-attacks/. Accessed 11 Jul 2024.
10. Natalucci, F., Qureshi, M. & Suntheim, F. Rising Cyber Threats Pose Serious Concerns for Financial Stability. *Imf.* https://www.imf.org/en/Blogs/Articles/2024/04/09/rising-cyber-threats-pose-serious-concerns-for-financial-stability. Accessed 11 Jul 2024 (2024).
11. Jelen, S. What are Brute Force Attacks? Definition, Types and Examples. https://securitytrails.com/blog/brute-force-attacks. Accessed 11 Jul 2024.
12. Idhom, M., Wahanani, H. E. & Fauzi, A. Network security system on multiple servers against brute force attacks. In *Proceeding - 6th Information Technology International Seminar, ITIS 2020* 258–262. https://doi.org/10.1109/ITIS50118.2020.9321108 (2020).
13. Yusup, M., Maisyaroh, M. & Septiana, L. Securing web application by embedded firewall at Gytech Indosantara Mandiri Ltd. *PIKSEL Penelit. Ilmu. Komput. Sist. Embed. Log.* **8**, 49–58. https://doi.org/10.33558/piksel.v8i1.2019 (2020).
14. Hassan, M. A. & Shukur, Z. A Secure multi factor user authentication framework for electronic payment system. In *2021 3rd Int. Cyber Resil. Conf. CRC 2021*. https://doi.org/10.1109/CRC50527.2021.9392564 (2021).
15. Riyaz Fathima, A. & Saravanan, A. An approach to cloud user access control using behavioral biometric-based authentication and continuous monitoring. *Int. J. Adv. Technol. Eng. Explor.* **11**, 1469–1496. https://doi.org/10.19101/IJATEE.2024.111100516 (2024).
16. Koilpillai, J. Software Defined Perimeter (SDP) 1 A Primer for CIOs. Waverley Labs (2017).
17. Kerman, A., Borchert, O., Rose, S. & Tan, A. Implementing a Zero Trust Architecture. Accessed 10 May 2021. Available: https://www.nccoe.nist.gov/sites/default/files/legacy-files/zta-project-description-final.pdf (2020).
18. Kumar, P., Moubayed, A., Refaey, A., Shami, A. & Koilpillai, J. Performance analysis of SDP for secure internal enterprises. In *IEEE Wirel. Commun. Netw. Conf. WCNC*. https://doi.org/10.1109/WCNC.2019.8885784 (2019).
19. Moubayed, A., Refaey, A. & Shami, A. Software-defined perimeter (sdp): State of the art secure solution for modern networks. *IEEE Netw.* **33**(5), 226–233 (2019).
20. Singh, J., Refaey, A. & Koilpillai, J. Adoption of the software-defined perimeter (SDP) architecture for infrastructure as a service. *Can. J. Electr. Comput. Eng.* **43**(4), 357–363. https://doi.org/10.1109/CJECE.2020.3005316 (2020).
21. Mcquaid, A. & Macdonald, N. Market Guide for Zero Trust Network Access. https://www.gartner.com/en/documents/4632099. Accessed 26 Mar 2025 (2022).
22. Fenton, J. L. et al. NIST Special Publication 800-63B Digital Identity Guidelines NIST Special Publication 800-63B Digital Identity Guidelines Authentication and Lifecycle Management 1–83. https://doi.org/10.6028/NIST.SP.800-63b (2023).
23. Rais, R., Morillo, C., Gilman, E. & Barth, D. *Zero Trust Networks: Building Secure Systems in Untrusted Network* 335 (Posts Telecommun. Press, 2024).
24. Garbis, J. & Chapman, J. W. *Zero Trust Security: An Enterprise Guide* (Apress, 2021).
25. Ward, R. & Beyer, B. BeyondCorp: A new approach to enterprise security. *Usenix* **39**, 6–11 (2014).
26. Cloud Security Alliance. Software-Defined Perimeter (SDP) Specification v2.0 2. Accessed 26 Mar 2025. Available: https://cloudsecurityalliance.org (2022).
27. Goerlich, J. W., Nather, W. & Pham, T. Zero Trust Going Beyond the Perimeter. https://resources.duo.com/explore/assets/zero-trust-going-beyond-the-perimeter. Accessed 26 Mar 2025 (2021).
28. Paloalto networks, White Paper, and www.paloaltonetworks.com. Architecting the Zero Trust Enterprise. Accessed 26 Mar 2025. Available: https://start.paloaltonetworks.com/2020-forrester-ztx-report?utm_source=social&utm_medium=blog&utm_campaign (2021).
29. Cozzolino, V., Schwellnus, N., Ding, A. Y., Ott, J. & Ding, A. Y. UIDS : Unikernel-based intrusion detection system for the internet of things. In *Work. Decentralized IoT Syst. Secur.* (2020).
30. Hossain, M. D., Ochiai, H., Doudou, F. & Kadobayashi, Y. SSH and FTP brute-force attacks detection in computer networks: Lstm and machine learning approaches. In *2020 5th International Conference on Computer and Communication Systems, ICCCS 2020* 491–497. https://doi.org/10.1109/ICCCS49078.2020.9118459 (2020).
31. Khashan, O. A. Secure outsourcing and sharing of cloud data using a user-side encrypted file system. *IEEE Access* https://doi.org/10.1109/ACCESS.2020.3039163 (2020).
32. Hall, R. C., Hoppa, M. A. & Hu, Y.-H. An empirical study of password policy compliance. *J. Colloq. Inf. Syst. Secur. Educ.* **10**, 8. https://doi.org/10.53735/cisse.v10i1.156 (2023).
33. Nadeem, M., Arshad, A., Riaz, S., Band, S. S. & Mosavi, A. Intercept the cloud network from brute force and ddos attacks via intrusion detection and prevention system. *IEEE Access* **9**, 152300–152309. https://doi.org/10.1109/ACCESS.2021.3126535 (2021).
34. Liu, Y., Squires, M. R., Taylor, C. R., Walls, R. J. & Shue, C. A. Account lockouts: Characterizing and preventing account denial-of-service attacks. In *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering, LNICST* Vol. 305, 26–46. https://doi.org/10.1007/978-3-030-37231-6_2 (2019).
35. Vizváry, M. & Vykopal, J. Flow-based detection of RDP brute-force attacks 1 Introduction 2 Design of the flow-based signature of RDP authentication. In *Proc. 7th Int. Conf. Secur. Prot. Inf.* 131–137 (2013).
36. Yang, X. et al. SecuDB: An in-enclave privacy-preserving and tamper-resistant relational database. *Proc. VLDB Endow.* **17**, 3906–3919. https://doi.org/10.14778/3685800.3685815 (2024).
37. Sekar, A. K., Ganesh, A., Ramakrishnan, R. & T. Kiruthiga, R. Emerging cyber security and brute force attacks in hospital management information systems. In *2023 2nd International Conference on Smart Technologies for Smart Nation, SmartTechCon 2023* 421–426. https://doi.org/10.1109/SmartTechCon57526.2023.10391825 (2023).
38. Jiang, Q., Zeadally, S., Ma, J. & He, D. Lightweight three-factor authentication and key agreement protocol for internet-integrated wireless sensor networks. *IEEE Access* **5**, 3376–3392. https://doi.org/10.1109/ACCESS.2017.2673239 (2017).
39. Chew, C. J., Wang, P. Y. & Lee, J. S. Behavioral analysis zero-trust architecture relying on adaptive multifactor and threat determination. *KSII Trans. Internet Inf. Syst.* **17**, 2529–2549. https://doi.org/10.3837/tiis.2023.09.012 (2023).
40. Hacking Multifactor Authentication. *Netw. Secur.* 2021, **4**. https://doi.org/10.1016/s1353-4858(21)00004-0 (2021). https://www.magonlinelibrary.com/doi/abs/10.1016/S1353-4858%2821%2900004-0
41. James, L. & Dileesh, E. D. Technique to thwart brute-force attack : A survey. *Int. J. Sci. Res. Sci. Eng. Technol.* **7**, 235–237. https://doi.org/10.32628/ijsrset207139 (2020).
42. Wu, Y., Cao, P. M., Withers, A., Kalbarczyk, Z. T. & Iyer, R. K. Mining threat intelligence from billion-scale SSH brute-force attacks. https://doi.org/10.14722/diss.2020.23007 (2020).
43. Cherry, D. *Enterprise-Grade IT Security for Small and Medium Businesses: Building Security Systems, in Plain English* (Apress, 2022).
44. Muhanad, A.-Z. & Huang Dongjun, M. K. DBFST: Detecting distributed brute force attack on a single target. *Int. J. Sci. Eng. Res.* **6**, 738–744 (2015).

45. Alotibi, N. & Alshammari, M. Deep learning-based intrusion detection: A novel approach for identifying brute-force attacks on FTP and SSH protocol. *Int. J. Adv. Comput. Sci. Appl.* **14**, 107–111. https://doi.org/10.14569/IJACSA.2023.0140612 (2023).
46. Hajjouz, A. & Avksentieva, E. Evaluating the effectiveness of the CatBoost classifier in distinguishing benign traffic, FTP BruteForce and SSH BruteForce traffic. In *2024 9th Int. Conf. Signal Image Process. ICSIP 2024* 351–358. https://doi.org/10.1109/ICSIP61881.2024.10671552 (2024).
47. Bonneau, J., Herley, C., Van Oorschot, P. C. & Stajano, F. Passwords and the evolution of imperfect authentication. *Commun. ACM* **58**(7), 78–87. https://doi.org/10.1145/2699390 (2015).
48. Saifan, R., Salem, A., Zaidan, D. & Swidan, A. A Survey of behavioral authentication using keystroke dynamics: Touch screens and mobile devices. *J. Soc. Sci.* **5**, 29–41. https://doi.org/10.25255/jss.2016.5.1.29.41 (2016).
49. Sasada, T., Taenaka, Y., Kadobayashi, Y. & Fall, D. Web-biometrics for user authenticity verification in zero trust access control. *IEEE Access* **12**, 129611–129622. https://doi.org/10.1109/ACCESS.2024.3413696 (2024).
50. Iță, C.-R., Constantinescu, R.-C., Vlădescu, A. & Alexandrescu, B. Security in remote access, based on zero trust model concepts and SSH authentication with signed certificates. https://doi.org/10.1117/12.2643058 (2023).
51. Petsios, T., Kemerlis, V. P., Polychronakis, M. & Keromytis, A. D. DynaGuard: Armoring canary-based protections against brute-force attacks. In *ACM International Conference Proceeding Series* 351–360. https://doi.org/10.1145/2818000.2818031 (2015).
52. Wang, K., Hong, Y., Li, Y., Yan, R. & Feng, J. A distributed zero-trust scheme for airborne wireless sensor networks using dynamic identity authentication. *Sci. Rep.* **15**(1), 1–29. https://doi.org/10.1038/s41598-025-91957-2 (2025).
53. Sallam, A., Refaey, A. & Shami, A. On the security of SDN: A completed secure and scalable framework using the software-defined perimeter. *IEEE Access* **7**, 146577–146587. https://doi.org/10.1109/ACCESS.2019.2939780 (2019).
54. WaverleyLabs. GitHub—WaverleyLabs/fwknop: Client and Gateway Modules for Software Defined Perimeter (SDP). https://github.com/WaverleyLabs/fwknop. Accessed 26 Mar 26 2025.
55. AifrruisLabs. GitHub—aifrruislabs/sdp-gateway-client: SDP Desktop and Gateway Clients. https://github.com/aifrruislabs/sdp-gateway-client. Accessed 26 Mar 2025 (2023).
56. AifrruisLabs. GitHub—aifrruislabs/sdp-controller: SDP Controller API based on Laravel and Vue JS. https://github.com/aifrruislabs/sdp-controller. Accessed 26 Mar 2025 (2023).
57. Hauser, V. Releases · vanhauser-thc/thc-hydra. https://github.com/vanhauser-thc/thc-hydra/releases. Accessed 26 Mar 2025.
58. nmap.org. Nmap: The Network Mapper—Free Security Scanner. https://nmap.org/. Accessed 26 Mar 2025 (2021).
59. Anna John, L. & Mathew, J. Efficient brute force attack handling: Server virtualization. In *Proc. Natl. Conf. Emerg. Comput. Appl.* Vol. 3, 218–221 (2021).
60. Ruambo, F. A., Zou, D., Lopes, I. & Yuan, B. Securing Sdn/Nfv-Enabled Campus Networks with Software-Defined Perimeter-Based Zero-Trust Architecture. https://doi.org/10.2139/ssrn.4511057 (2023).

Acknowledgements

Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2025R752), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia. This paper also derived from a research grant funded by the Research, Development, and Innovation Authority (RDIA)—Kingdom of Saudi Arabia—with grant number (13325-psu-2023-PSNU-R-3-1-EF). Additionally, the authors would like to thank Prince Sultan University for their support.

Author contributions

F.A.R.: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Original Draft Preparation, Writing—Review & Editing, Visualization, Project Administration, Funding Acquisition. E.E.M.: Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Review & Editing. B.L.: Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Review & Editing. A.A.A.: Conceptualization, Validation, Data Curation, and Writing—Review & Editing. A.A.A.E.: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Review & Editing, Supervision, Project Administration, Funding Acquisition. M.A.: Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing—Review & Editing. B.A.: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, and Writing—Review & Editing.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to A.A.A.-L. or B.A.-E.-A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025